National Library of Canada
Bibliothèque nationale du Canada

Canadian Theses Service
Service des thèses canadiennes

Ottawa, Canada
K1A 0N4

NOTICE

The quality of this microform is heavily dependent upon the quality of the original thesis submitted for microfilming. Every effort has been made to ensure the highest quality of reproduction possible.

If pages are missing, contact the university which granted the degree.

Some pages may have indistinct print especially if the original pages were typed with a poor typewriter ribbon or if the university sent us an inferior photocopy.

Reproduction in full or in part of this microform is governed by the Canadian Copyright Act, R.S.C. 1970, c. C-30, and subsequent amendments.

AVIS

La qualité de cette microforme dépend grandement de la qualité de la thèse soumise au microfilmage. Nous avons tout fait pour assurer une qualité supérieure de reproduction.

S'il manque des pages, veuillez communiquer avec l'université qui a conféré le grade.

La qualité d'impression de certaines pages peut laisser à désirer, surtout si les pages originales ont été dactylographiées à l'aide d'un ruban usé ou si l'université nous a fait parvenir une photocopie de qualité inférieure.

La reproduction, même partielle, de cette microforme est soumise à la Loi canadienne sur le droit d'auteur, SRC 1970, c. C-30, et ses amendements subséquents.

NL-339 (r. 88/04) c

Canada

University of Alberta

GETS: A Natural Language Generator for Specifications

by

Abul K. Salek

A thesis
submitted to the Faculty of Graduate Studies and Research
in partial fulfillment of the requirements for the degree
of Master of Science

Department of Computing Science

Edmonton, Alberta
Fall, 1991

National Library of Canada
Bibliothèque nationale du Canada

Canadian Theses Service
Service des thèses canadiennes

Ottawa, Canada
K1A 0N4

The author has granted an irrevocable non-exclusive licence allowing the National Library of Canada to reproduce, loan, distribute or sell copies of his/her thesis by any means and in any form or format, making this thesis available to interested persons.

L'auteur a accordé une licence irrévocable et non exclusive permettant à la Bibliothèque nationale du Canada de reproduire, prêter, distribuer ou vendre des copies de sa thèse de quelque manière et sous quelque forme que ce soit pour mettre des exemplaires de cette thèse à la disposition des personnes intéressées.

The author retains ownership of the copyright in his/her thesis. Neither the thesis nor substantial extracts from it may be printed or otherwise reproduced without his/her permission.

L'auteur conserve la propriété du droit d'auteur qui protège sa thèse. Ni la thèse ni des extraits substantiels de celle-ci ne doivent être imprimés ou autrement reproduits sans son autorisation.

ISBN 0-315-69946-9

Canada

UNIVERSITY OF ALBERTA
RELEASE FORM

NAME OF AUTHOR: Abul K. Salek
TITLE OF THESIS: GETS: A Natural Language Generator for Specifications
DEGREE: Master of Science
YEAR THIS DEGREE GRANTED: 1991

Permission is hereby granted to University of Alberta Library to reproduce single copies of this thesis and to lend or sell such copies for private, scholarly or scientific research purposes only.

The author reserves all other publication and other rights in association with the copyright in the thesis, and except as hereinbefore provided neither the thesis nor any substantial portion thereof may be printed or otherwise reproduced in any material form whatever without the author's prior written permission.

(Signed) ..A.K.Salek.. ...
Permanent Address:
24-13, New Colony
Mohammadpur
Dhaka 1207
Bangladesh

Dated September 16, 1991

UNIVERSITY OF ALBERTA

FACULTY OF GRADUATE STUDIES AND RESEARCH

The undersigned certify that they have read, and recommend to the Faculty of Graduate Studies and Research for acceptance, a thesis entitled **GETS: A Natural Language Generator for Specifications** submitted by **Abul K. Salek** in partial fulfillment of the requirements for the degree of **Master of Science.**

Dr. Paul Sorenson, Supervisor

Dr. Jim Hoover, Committee Member

Dr. Peter van Beek, Committee Member

Dr. Ming Rao, Committee Member

September 16, 1991

To my parents

ABSTRACT

An architectural design is developed for a natural language generator that is capable of producing text from a formal specification in Telos. Formal specifications are difficult to understand by clients of information systems due to the complexity and formality of the specification language. Our system is intended to help the client better understand the specification by providing a natural language report that paraphrases the specification. The system, known as Generator of English Text from Specification (GETS), can generate complete and partial reports. Algorithms for content selection and discourse strategy are provided. Also given are the strategies to generate sentences from an object definition.

The problem of paraphrasing a specification in an object-oriented paradigm has not been examined previously. This thesis scrutinizes this problem and proposes a solution for it.

Acknowledgements

I would like to thank my supervisor, Dr. Paul Sorenson, for his continuous guidance of my research and meticulous reading of this thesis. I would also like to thank the members of my committee: Dr. Peter van Beek, Dr. Jim Hoover, and Dr. Ming Rao for carefully reading this thesis and making positive criticisms. A special thanks goes to Dr. Germinal Boloix, and other members of the Software Engineering Research Group for all the discussions I had with them during my research work. I take this opportunity to acknowledge the financial assistance of the Department of Computing Science. I am also indebted to Dr. Sorenson for providing me funding support from his NSERC operating grant.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Scope and Overview of the Thesis . . . 2

2 Background **4**
- 2.1 Natural Language Generation Systems in the 1960's and 1970's . . . 5
- 2.2 Natural Language Generation Systems in the 1980's . . . 8
  - 2.2.1 MUMBLE . . . 8
  - 2.2.2 TEXT . . . 9
  - 2.2.3 Penman . . . 10
  - 2.2.4 PAULINE . . . 12
- 2.3 Generation from Predicate Calculus . . . 16
- 2.4 Generation from Specification . . . 17
  - 2.4.1 Example GIST Specifications . . . 18
- 2.5 Conclusion . . . 21

3 An Overview of Telos **22**
- 3.1 Introduction to Telos . . . 22
- 3.2 Propositions . . . 23
- 3.3 Structuring Mechanisms . . . 24
  - 3.3.1 The Instantiation Hierarchy . . . 25
  - 3.3.2 The IsA Hierarchy . . . 25
  - 3.3.3 Examples of Hierarchy . . . 26
- 3.4 Representation of Temporal Knowledge . . . 31
- 3.5 Assertions in Telos . . . 33
- 3.6 Meta-Definitions . . . 36
- 3.7 Other Features . . . 37

4 Sentence Generation **38**
- 4.1 Sentence Generation Mechanism . . . 38
- 4.2 Identification Sentences . . . 41
  - 4.2.1 Examples of Generation from Introductory Component . . . 44
    - 4.2.1.1 Example 4.1 . . . 45
    - 4.2.1.2 Example 4.2 . . . 46
- 4.3 Attribute Descriptions . . . 47
  - 4.3.1 Abstract Objects . . . 47
  - 4.3.2 Example of Generation from Attribute Component . . . 52
  - 4.3.3 Tokens . . . 54
- 4.4 Incorporating Temporal Expressions . . . 56
- 4.5 Sentences from Assertions . . . 57
- 4.6 Architecture: Sentence Generator . . . 62
  - 4.6.1 Recognizer . . . 62
  - 4.6.2 Arranger . . . 63

4.6.3 PSG . . . 63
4.6.4 Local Refiner . . . 63
4.6.5 Global Refiner . . . 64
4.7 Conclusion . . . 65

5 Discourse Strategies **66**
5.1 Data Flow . . . 66
5.2 Input Processor . . . 68
5.3 Content Selector . . . 69
5.3.1 Full Report . . . 69
5.3.1.1 Membership *or* Specialization Link Reduction Heuristic . . . 72
5.3.1.2 Membership *and* Specialization Link Reduction Heuristic . . . 74
5.3.1.3 Token Link Reduction Heuristic . . . 75
5.3.2 Example of a Graph Traversal: . . . 80
5.3.3 Example of a Full Report . . . 81
5.3.4 Selective Report . . . 84
5.3.5 Example of a Selective Report . . . 85
5.3.6 Data Report . . . 85
5.4 The Output Processor . . . 86
5.5 Conclusion . . . 87

6 Comparisons **88**
6.1 Comparison with GIST Generator . . . 88

7 Conclusion **92**
7.1 Summary and Conclusion . . . 92
7.2 Future Research . . . 95

8 Bibliography **98**

List of Tables

3.1 Levels of classes in Telos. 25
3.2 Built-in classes in Telos. 26

4.1 Translations of Temporal Constraints. 57

List of Figures

2.1 The identification schema [McKeown, 1985] 10
2.2 Architecture of TEXT [McKeown, 1985] 11
2.3 Overview of Penman [Mann, 1983] 12
2.4 Architecture of PAULINE [Hovy, 1990] 15

3.1 Object Hierarchies of a Department Information System 27
3.2 Semantic Net Representation of Department Information System . . . 30
3.3 Temporal Relations with their Inverses [Koubarakis et al., 1989a] . . 32

4.1 Components of the Sentence Generator of GETS 40
4.2 Hierarchies for Employee, Researcher and Faculty 43

5.1 Data flow in GETS 67
5.2 Membership Link 70
5.3 Specialization Link 70
5.4 Token Link 71
5.5 Referential links 71
5.6 Algorithm for Graph Generation 73
5.7 Marking links Inactive: 1 74
5.8 Marking links Inactive: 2 74
5.9 Marking links Inactive: 3 75
5.10 Algorithm for Graph simplification 76
5.11 Algorithm for Graph Traversal 78
5.12 An Example of depth- and breadth-queues 79
5.13 Procedure Visit 79
5.14 A graph for traversal 80
5.15 Snapshot of Data During Execution of Alg. Graph_Traversal 82
5.16 Graph generated from Figure 3.1 and its simplification 83
5.17 A graph from which a selective report is generated 85

Chapter 1

Introduction

This thesis examines the problem of paraphrasing in English all or part of a formal specification, in particular a specification written in Telos. Very little work has been done in the translation of specifications, especially those based on an object-oriented paradigm. The major contributions resulting from our work are the development of content selection, discourse and sentence generation strategies that apply to object-oriented approaches to specification.

An explanation of the importance of such report generation is motivated in this chapter. The scope of our research and the organization of the thesis is then described.

1.1 Motivation

Requirement modeling and analysis is an important, initial step in any system development process. Often the requirement analysis results in a formal specification of the system. A specification reveals detailed, precise and complete requirements that the client wants for his/her system. The specification not only helps in capturing the functionality of the system, but also plays important roles in system design, verification, validation and evaluation. A specification provides system documentation in support of information exchange between the client and the analyst, and between the analyst and the implementor. More importantly, a specification serves as a 'contract' between the client and the system developer. The advantages of a formal specification

and a survey of several formal languages are given in [Wing, 1990].

Formal specifications require sound mathematical foundations. Specification languages are typically based on set theory (e.g., the specification language, Z), axioms (e.g., Iota, OBJ, Anna etc.), algebras (e.g., Clear, Act One etc.) or both algebras and axioms (e.g., Larch). Many specification languages are also based on first-order predicate calculus (e.g., Ina Jo [Kemmerer, 1990]), and object-oriented formalism (e.g., Telos [Mylopolous et al., 1990]).

Whatever the foundation of a specification language is, it is evident that they tend to be technical and complicated. In addition, the complexity of a system specification increases as the size of the system increases. For example, in a large Telos specification there may be hundreds of object definitions. The relationships among objects and the applicable restrictive rules on them may not be obvious by just browsing through the specification knowledge base. A vast Telos specification may be 'confusing' to the system developer, let alone the client who is less likely to be familiar with the syntax and semantics of Telos.

The goal of our research is to generate a natural language explanation of a Telos specification. The natural language description generated is a report on the whole or part of the specification. This report is easily comprehensible by both the client and analyst, and may be used as a contract between the client and the system developer. This report can serve as a documentation of the system's functionality and help in all system development phases. At a later phase of development, if any ambiguity arises, it may be easier for the analyst to refer to the natural language report rather than the formal specification.

1.2 Scope and Overview of the Thesis

We will give an architectural design of a system that can generate several types of reports based on user request. Our research is directed primarily to the client as a potential user of the system. Since Telos contains first-order predicate formulae

in addition to its object-definition framework, we address translation of both the object-definition part as well as predicate logic formulae – although the latter may be incomplete because of the complicated nature of Telos assertions.

Telos provides a novel knowledge representation mechanism based on semantic nets and object-oriented schema, and is suitable for knowledge base construction, formal specification and conceptual modeling. Although our approach is based on Telos, it should be general enough to be transported to other object-oriented systems with minimal modification.

Our syatem GETS is currently being implemented in C++ to run under the Unix operating system. It is being implemented in cooperation with an NSERC summer student working for the Software Engineering Group.

Chapter 2 of this thesis provides a survey of natural language generation systems. Chapter 3 presents an overview of Telos. Chapter 4 considers the strategies of generating English sentences from an object definition while Chapter 5 discusses procedures of content selection. In Chapter 6, we compare our system with GIST Generator, another natural language generator. We choose the GIST Generator for comparison because of its similarity in purpose. It also operates on formal specifications. A summary of our research, conclusions and an outline of some future research issues are given in the concluding chapter.

Chapter 2

Background

Research in natural language generation has received considerably less attention than the complementary process of natural language analysis. Although they share some common methodology, generation is not simply the reverse process of analysis. The analysis process follows the stages of morphological, syntactic, semantic, pragmatic and discourse analyses; whereas the generation process follows the stages of identification of goals, content selection, rhetorical planning and surface syntactic construction [McDonald, 1987]. At the first step of generation, a goal is formulated to determine the intention of the particular text to be generated. The goal guides the generator to select the content of the text. The planning is necessary to determine how the text is conveyed. Finally, the syntactic construction is performed to create the actual sentences.

Some of the template-based generation systems of the 1970's produced impressive sentences. They gave the impression that generation was a simple task and this was one of the reasons why significant research in generation were late to develop. It was not until the 1980's that AI-based approaches were used to solve the problems of generation. In this chapter the evolution of research in language generation is traced. The discussion is not a detailed one because none of these systems work on an object-oriented framework that is the focus of this thesis. For a detailed survey and elaborate discussion of some of these systems, please refer to [Bailes, 1986].

2.1 Natural Language Generation Systems in the 1960's and 1970's

The earliest generation systems dates back to the 1960's. They were simple with limited capabilities and sophisticated discourse and pragmatic considerations were not present.

Simmons' SYNTEX [Simmons, 1965] retrieved information from a database of English text on which an inverted index was created. After a brief syntactic analysis the system followed the index to refer to the appropriate part of the text and generated an answer. The problem with this system was that human interventions were often needed to resolve ambiguities.

ELIZA (or DOCTOR as it is sometimes known) by [Weizenbaum, 1966] modeled psychiatric consultation of a patient with a psychiatrist. It matched templates against the user's input sentences and generated responses. The response templates were stored under keywords. Even though the system was robust it often generated incoherent sentences when it failed to properly match user input.

Winograd's SHRDLU (1972) was one of the earlier generation systems that has received wide attention. The system operated on a block world which had several blocks of different sizes, shapes and colors on a table. The semantic analyzer of SHRDLU mapped the output of the syntactic tree to a language called MICROPLANNER which worked on an assertional database. The system was capable of generating patterned responses, answers to questions, descriptions of objects and events, and a fluent discourse [Winograd, 1972]. The patterned responses were built-in or template driven. SHRDLU classified various types of questions such as WH-questions, HOW-MANY questions, YES-NO questions etc. It would generate appropriate answers for different categories of questions. In order to describe objects and events it first examined the database and found relevant information about the asked objects. Then the answer was generated by instantiating a set of patterns corresponding to

those objects. SHRDLU also used heuristics to resolve references and appropriate pronominalization.

The following examples exhibit some of the responses SHRDLU generated [Winograd, 1972]:

This example has been taken out for copyright restrictions.
The example shows the responses of SHRDLU.
It is available in [Winograd, 1972] (see bibliography)

It should be noted that this template-instantiation approach was used in the explanation components of two expert systems MYCIN [Scott et al., 1977] and Digital Therapy Adviser [Swartout, 1979, 1981].

Simmons and Slocum developed a system based on Augmented Transition Network (ATN) grammar which was one of the earliest grammar-based generation systems [Simmons & Slocum, 1972]. The input to the system was a semantic network that captured the propositional contents of the sentences to be generated. The elements of a semantic net were *concepts* and *relations* where a concept was a wordsense, and a relation was either an intersentential connectives such as *since, thus* etc., or a deep case relations such as agent, object or dative. In their system, the lexicon was also represented in semantic net form beside discourse.

The generator applied a series of transformation functions in an order determined by the grammar network and modified a semantic structure successively. At first a node was selected from the net and the generator entered the grammar at the path corresponding to that node. Then a set of form-determiners was selected. These form-determiners included voice, form, aspects, tense, and mood. Once it had determined

the path in the network a simple traversal of an arc produced a clause, a phrase, or a complete sentence.

Simmons and Slocum's technique was simple but effective; however, it fell short of a complete discourse generator. The simple pattern matching with user's question employed in their system, was not adequate for determining the complete discourse of an utterance.

Wong developed a methodology to generate sentences from semantic nets which was used in the TORUS system [Wong, 1975]. His method eliminated some limitations of discourse, and the problems associated with pronominalization, determiner and case placement in Simmons and Slocum's approach. Some other semantic network based approaches were described in [Shapiro, 1981] and [Boyer & Lapalme, 1985, 1985a]. Shapiro used a generalized augmented transition network grammar formalism, and Boyer and Lapalme provided a mechanism with which text can be associated with meaning with the help of unification.

Based on the *conceptual dependency* of Schank (1975), Goldman developed a generation system that was a component of MERGIE, a natural language understanding system [Goldman, 1975]. He used a *discrimination net* (DN) structure to organize word choices. A DN is a binary tree with its nodes as predicates and leaves as functions. The truth value of a predicate determines which branch is chosen next. During this decision making poor lexical choices get filtered out. The functions are used to build the input to the generator.

Goldman's system eliminated the problem of re-interpretation of the original meaning that was lost in the translation to a deep meaning representation. Even though Goldman's system had certain interesting features such as capability of multiple paraphrasing it did not give solutions to the discourse problems that the system of Simmons and Slocum had.

Davey's program PROTEUS, developed in 1974 [Davey, 1979] produced descriptions of the game tic-tac-toe that was considered as one of "the most fluent texts

ever produced by a machine" [McDonald, 1987]. Equipped with a rich conceptual model of the game it could identify the *threatening* and *blocking* moves as well as missed opportunities or forks. The text planner would group two or three moves which would be revealed as English sentences by a systemic grammar facility. An output of PROTEUS is given below:

> "The game started with my taking a corner, and you took an adjacent one. I threatened you by taking the middle of the edge opposite that and adjacent to the one which I had just taken but you blocked it and threatened me. I blocked your diagonal and forked you. If you had blocked mine, you would have forked me, but you took the middle of the edge opposite of the corner which I took first and the one which you had just taken and so I won by completing my diagonal."

In 1974 Clippinger developed ERMA [Clippinger, 1977], a program that modeled the real-time speech of people where thinking and planning are done simultaneously. A talking human being often pauses in the midsentence and replans the utterance. ERMA tried to capture this phenomena and produced a paragraph of speech by a psychoanalytic patient. Unfortunately, because of the computational complexity the project was short-lived [McDonald, 1987].

2.2 Natural Language Generation Systems in the 1980's

Research in natural language generation experienced a stagnation in the late 1970's; however, considerable work was conducted in the early 1980's. A number of Ph.D. dissertations were completed [McDonald, 1980] [Appelt, 1981] [Swartout, 1981] [McKeown, 1982] [Tait, 1982] [Conklin, 1983] [Kukich, 1983, 1988] [Woolf, 1984] [Jacobs, 1985, 1987] [McCoy, 1985] [Hovy, 1987, 1988] [Watt, 1988 - Master's thesis]. In this section we will discuss some of the major systems developed in the last decade.

2.2.1 MUMBLE

MUMBLE [McDonald, 1987a] uses a multi-level, description-directed approach towards language generation. It is multi-level because it operates on three levels of

representation: message, surface structure and word stream. It also uses a description-directed control protocol in which data in the above three levels are interpreted as instructions to the generator.

MUMBLE has three processes that are interleaved to handle transitions between the three levels of representations:

a) *Phrase Structure Execution:* This process is responsible for interpreting surface structure, defining active grammatical constraints and producing output word stream.

b) *Attachment:* This is responsible for interpreting messages and transferring message components to appropriate positions in the surface structure.

c) *Realization:* This selects from alternative forms to generate surface structure phrases.

The template-based approach fails to maintain "an adequate representation of grammatical context" [McDonald, 1987a] which is overcome by the message-directed system such as MUMBLE.

2.2.2 TEXT

McKeown's system TEXT [McKeown, 1982, 1982a, 1983, 1984, 1985, 1985a] generates paragraph-length answers in response to questions about database structures. It uses a formal computational model of *discourse strategy* and a set of constraints known as *focus of attention*, both of which had not been used previously. The discourse strategy selects the content from the knowledge base and orders it, while focus determines which object to describe and what will be the best utterance.

TEXT uses *rhetorical predicates* to determine discourse. It identifies the following four predicate patterns and represents them as *schemata*: identification, constituency, attributive, and contrastive schemata. Figure 2.1 gives an example of a schema and its relation to TEXT. The schemata were implemented on an ATN-based formalism.

The generation is a two phase process in TEXT: the *strategic* part that determines the content and the flow of content, and the *tactical* part that realizes the output of

Figure 2.1 has been taken out for copyright restrictions.
This figure shows the identification schema of TEXT.
It is available in [McKeown, 1985] (see bibliography)

the strategic part into text with the help of a functional grammar [Kay, 1979] and a lexicon.

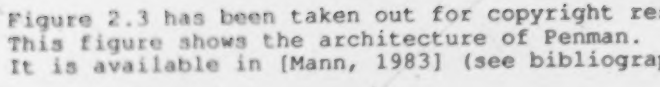
The architecture of TEXT is given in Figure 2.2. The user input is interpreted to find out what type of response is to be generated and the relevant knowledge pool is gathered related to the request. Then a discourse strategy is chosen and appropriate propositions are selected. Finally, the tactical component transforms the message generated from the knowledge pool into text. The contribution of TEXT is in focus constraint and discourse strategies of language generation.

The knowledge-based generator, GENNY, uses rhetorical predicates and focus mechanism, analogous to TEXT, and utilizes pragmatic information in text generation [Maybury, 1989].

2.2.3 Penman

Penman [Mann, 1983, 1983a, 1984, 1987] was designed to produce fluent multiparagraph English text. The semantics are independent of a system's knowledge domain giving it the freedom to be used with any knowledge representation. The system has four major modules: knowledge acquisition module, text planning module, sentence

Page 11 (Figure 2.2) has been taken out for copyright restrictions.
This figure shows the architecture of TEXT.
It is available in [McKeown, 1985] (see bibliography)



generator, and evaluation and plan-perturbation module as depicted in Figure 2.3.

The knowledge acquisition module gathers relevant information by domain-specific searches. The text planning module organizes the gathered information for presentation and plans how to lead the readers attention through the content of the text produced. Penman uses a hierarchical successive-refinement planner which is based on a theory of discourse and a model of the reader.

The sentence generator uses a systemic grammar called Nigel [Mann, 1983b] [Mann & Matthiessen, 1983]. Instead of using production rules, a systemic grammar uses units known as *systems* which are a collection of alternatives called *grammatical features*. [Mann, 1982] describes how systemic choices are made.

The evaluation module appraises the quality of the text and tries to improve it by producing perturbation in the plan.

The novel features of Penman is that it uses a systemic grammar and considers text improvement techniques.

2.2.4 PAULINE

PAULINE [Hovy, 1985, 1988, 1988a, 1990] is a natural language generator that uses the same underlying input representation to generate different sentences depending

on the pragmatic aspects of the utterance.

In light of pragmatic effects PAULINE categorizes three groups of considerations:

This portion has been taken out for copyright restrictions.
This part shows how PAULINE categorizes pragmatic effects.
It is available in [Hovy, 1990] (see bibliography)

Figure 2.4 gives PAULINE's overall architecture. While translating the communicative goals of the speaker to syntactic forms, PAULINE uses some intermediate *rhetorical goals* of *opinion* and *style*. The pragmatic aspects as described above interact with the rhetorical goals in choosing appropriate strategies.

Traditionally, two phases of language generation, namely text planning and text realization, have been done either in a *separate* or an *integrated* approach. Both of these approaches have shortcomings. Therefore, PAULINE uses a *limited commitment* approach in which the planning process is deferred until it is needed by the realization process. This interleaving does not allow it to perform top-down planning alone; instead, PAULINE uses two types of planning: *prescriptive* and *restrictive* planning.

Prescriptive planning is long range planning that performs formative activities such as topic gathering, and paragraph organization.

Page 15 (Figure 2.4) has been taken out for copyright restrictions.
This figure shows the architecture of PAULINE.
It is available in [Hovy, 1990] (see bibliography)

The short range restrictive planning accomplishes activities such as alternative selections. The options are provided by the text realizer. It also handles conflicting goals by appropriately resolving them in a least-satisfied strategy.

The input to the system is a case-frame-type representation based on conceptual dependency.

The contribution of PAULINE is that it makes use of extensive pragmatic considerations.

2.3 Generation from Predicate Calculus

In language analysis, predicate calculus has been found to be a suitable semantic representation in many cases, and some work has been done on translating from English sentences to logical forms (such as, in [Sandwall, 1971]). The generation process can be viewed as the reverse translation from a logical form to an intermediate representation such as a transformational deep structure, and then from deep structure to surface structure, i.e., sentences. But the problem is the input may not be in an appropriate logical form for the translation process, and it is not easy to transform any logical form to the required appropriate form [Grishman, 1986].

Chester's program EXPOUND [Chester, 1976] translates formal proofs into English. Some lexical information such as verb strings, syntactic types (e.g., noun, adjective, phrase, clause etc.), preposition and gender is attached to each predicate. The generator identifies which subformulae generate noun phrases, verb phrases etc. and 'glues' them together with adjectives followed by nouns, followed by prepositional phrases, followed by dependent clauses to make a sentence. Discourse is determined with a partial ordering of the steps of the proof. Appropriate sentence connectives (thus, therefore, etc.) are added to enhance readability of the text.

McDonald has also done some work on translating a logical proof to natural language [McDonald, 1983]. It is different from Chester's system because the latter generates one sentence for each proof step. The former operates on the logic of the

whole proof and hence, may skip a few steps (e.g., a tautological step). Moreover, some proof steps may generate more than one sentence in McDonald's system to make the text more readable.

Edgar's system EXPLAIN [Edgar, 1991] translates a natural deduction proof of an automated theorem prover in four dimensions. The first dimension explains the strategy of the proof construction while the second deals with the amount of explanation done at each step. The third dimension concerns the abstraction of the explanation and the fourth dimension considers how the proof tree is traversed; in a bottom-up, top-down or a mixed fashion.

Shieber and others [Shieber et al., 1990] give an algorithm to generate strings, in a semantic-head-driven fashion, which allows wide classes of grammars including semantically non-monotonic grammar.

2.4 Generation from Specification

Swartout's GIST Generator [Swartout, 1982] produces English descriptions from a GIST specification. A GIST specification consists of a set of attribute relations, agents and actions. For a description of GIST see [Balzer et al., 1982]. The generator makes three passes over the specification to produce a description of the system. The first pass creates a case grammar from a GIST specification. The second pass improves the quality of the text by performing some inter-sentence transformations over the case grammar. And the third pass performs some intra-sentence optimizations and produces text from the grammar [Swartout, 1982].

Swartout later developed a natural language explainer of the execution trace of the symbolic evaluator of GIST. The symbolic evaluator was designed to capture the implicit information held in the specification [Swartout, 1983].

In chapter 6 we will compare our system to the GIST Generator. Both the GIST Generator and our system perform similar actions of translating a formal language into English. To illustrate the syntax and semantics of GIST language, we give some

example specifications in the next subsection.

2.4.1 Example GIST Specifications

Examples in this subsection regarding GIST have been taken from [Swartout, 1982]. A GIST specification contains a set of type declarations, agents, actions, and constraints. In the following, we discuss them briefly.

Each actions has an action-name with parameters, preconditions and the definition of the action.

Type declaration has a type-name with parameters. Each parameter denotes attribute relations to that type.

For example, the type declaration

type ship(Destination|port);

defines a type ship with one parameter. The parameter has an attribute relation Destination that identifies a binary relation between ship and port.

GIST offers two operations on these relations:

1) forward mapping: The operation :Destination is a forward mapping from Ships to ports, i.e., ship:Destination means *the destination of a ship.*

2) reverse mapping: The operation ::Destination is a reverse mapping from ports to ships, i.e., port::Destination means *a ship whose destination is the port.*

Agents also have the same syntactic structures as that of type declarations.

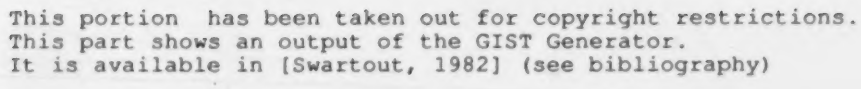
Below we give a complete GIST specification illustrating different aspects of the language.

This portion has been taken out for copyright restrictions.
This part shows an example of a GIST specification.
It is available in [Swartout, 1982] (see bibliography)

The above specification is not easy to understand because of its syntax as well as semantics associated with it. The structure of constraints, e.g., Berths_Are_In_Ports, and actions are specifically complicated.

The natural language translation of the above, generated by the GIST Generator,

is as follows:



As evident from the above text, GIST Generator has a simple discourse technique. The types, agents and actions are described as they appear. Constraints are moved at the end, e.g., Fuel and Grain and Berths Are In Ports are described later. The first sentence describes the major entities and another sentence mentions the actions

available right before explaining them.

2.5 Conclusion

This chapter discussed a number of language generators which fall into three categories: general generators, generators from predicate calculus and generators from specification. As apparent from the discussion, research in generation in the domain of specification is underdeveloped. The general generators are based on semantic net or some other internal representations designed for language generator, but not on a general object-oriented schema. We focus on giving a solution for the problem of language generation from an object-oriented system in the domain of specification.

Chapter 3

An Overview of Telos

In this chapter we present the syntax and semantics of Telos, the specification language used as a basis of our natural language generation system. The focus is on object definition, temporal constraints and assertions. Other aspects of Telos such as meta-attribute definitions and query and update commands are briefly described or avoided because they play little role in natural language generation by our current system.

3.1 Introduction to Telos

Telos is a requirements modeling language for software specification that is intended to support information system development. In addition, it is a knowledge representation language that captures knowledge about the environment and a system's interaction with that environment, knowledge about the type of information the system is supposed to handle, knowledge about the design and implementation of the system, design decisions and their justifications, and information on the software development process [Mylopoulos et al., 1990]. Telos provides mechanisms for representing concepts in a semantic net and frame-based object-oriented framework. It also has facilities for knowledge base construction, temporal reasoning and inference capabilities for query evaluation and integrity checking.

The ancestor of Telos is the Requirement Modeling Language, RML, developed

by Greenspan [Greenspan, 1984] [Greenspan et al., 1986]. RML, too, is based on an object-oriented framework with three classes of objects: *entities*, *activities* and *assertions*. Entity classes have attributes categorized under *producer*, *updater* and *consumer* and activity classes have attributes categorized under *input*, *output* and *control*. RML supports classification, generalization and aggregation, and treats time in a linear time-point model.

Some limitations in representation in RML led to the evolution of the Conceptual Modeling Language, CML [Borgida et al., 1985] [Stanley, 1986]. It provides a better representation of knowledge including temporal knowledge and meta-concepts. Telos is primarily a refinement of CML.

The implementations of CML are described in [Gallagher and Solomon, 1986], [Koubarakis, 1988] and [Jarke et al., 1988]. For implementations of Telos, see [Topaloglou and Koubarakis, 1989], [Eherer et al., 1989, 1990] and [Topaloglou, 1989].

Telos has been described in detail in [Mylopoulos et al., 1990], [Koubarakis et al., 1989, 1989a], [Topaloglou and Koubarakis, 1989] and [Topaloglou, 1989]. In the following sections we will briefly discuss the features of Telos.

3.2 Propositions

The representational framework of Telos is based on semantic nets, semantic data models and frame-based object-oriented schemata. *Propositions* are the building blocks of a Telos knowledge base. A proposition is represented as a quadruple <*source*, *label*, *destination*, *duration*> that designates the source, label-tag, destination and time duration of the proposition. Propositions are either *individuals* (in other words, entities, nodes, objects, concepts) or *attributes* (alternately, relationships, links). The components of the proposition are also propositions.

Example 3.1:

The fact `John is the president of the computer club for 1990` *can be represented in the following proposition:*

<John, president, computer club, 1990>

The built-in functions from(p), label(p), to(p) and when(p) return the source, label, destination and duration of the argument proposition p respectively.

Individuals are divided into three categories as follows:

a) *Simple Individuals* are those entities who have the source and destination as themselves. For example, John in the above is actually,

<John, ... , John, ...>

b) *Labels* are those individuals that have source, label and destination components as themselves. For example, the label president is represented as:

<president, president, president, ...>

c) *Time Intervals* are special individuals with source, destination and duration components as themselves. For example, the duration 1990 is represented as:

<1990, ..., 1990, 1990>

3.3 Structuring Mechanisms

The structuring mechanisms of Telos are as follows: *classification/instantiation, aggregation/decomposition* and *generalization/specialization.* The knowledge base is informed of an object with a TELL command as shown in Examples 3.2 and 3.3 below:

Example 3.2:

```
IndividualClass Student IN S_Class
WITH
    necessary, single
                id              : Integer
    attribute
                name            : String;
                credits         : Integer;
                academicAdvisor: Faculty;
                academicLevel   : Year
END Student
```

Example 3.3:

```
Individual Shirley IN Student, Token
WITH
    necessary, single
```

```
                id                : 904381
        attribute
                name              : Shirley Bates;
                credits           : 102;
                academicAdvisor: Paul Sorenson;
                academicLevel     : Senior
    END Shirley
```

Example 3.2 says that the class **Student** is an instance of the class **S_Class** (built in) with a number of attributes delineated with a **WITH** clause. The attributes are enumerated as *<label>:<type>*, label being optional, and are grouped into attribute categories. Example 3.2 says that **id** is a necessary attribute that must be unique. The other attributes of **Student** are **name**, **credits**, **academicAdvisor** and **academicLevel**. Example 3.3 **TELL**s the knowledge base about a specific student (ground instance) **Shirley**, identified by the keyword **Individual**.

3.3.1 The Instantiation Hierarchy

The ground instances are known as *tokens*. Objects having only tokens as instances are *simple classes* and objects having simple classes as instances are *metaclasses*. Classes are divided into the levels shown in Table 3.1.

The built-in classes of Telos have been shown in Table 3.2.

3.3.2 The IsA Hierarchy

Telos offers specialization/generalization mechanisms through the IsA hierarchy. Every class is a specialization of the built-in class of the same level. For example, in a

Table 3.1: Levels of classes in Telos.

Level	Includes
0	tokens
1	simple classes
2	m1_classes
3	m2_classes
...	...
w	w_classes

Table 3.2: Built-in classes in Telos.

Classes	Type	Includes
Token	simple class	all tokens
S_Class	m1_class	all simple classes + Token
M1_Class	m2_class	all m1_classes + S_Class
...	...	...
Proposition	w_class	all propositions + itself
Class	w_class	all classes + itself
Individual	w_class	all individuals + itself
Attribute	w_class	all attributes
IndividualClass	w_class	all individual classes + itself
AttributeClass	w_class	all attribute classes + itself
OmegaClass	w_class	all w_classes + itself

department information system, faculty is a specialization of the built-in class S_Class. The specialized class inherits the attributes from its parent generalized class.

Among the *ω_classes* AttributeClass is a specialization of Class and Attribute; Class, Attribute and Individual are specializations of Proposition; IndividualClass is a specialization of Class; Individual is a specialization of IndividualClass and OmegaClass is a specialization of IndividualClass.

3.3.3 Examples of Hierarchy

Figure 3.1 shows part of the Telos objects in a Department Information System. Single arrows denote instantiation while double arrows denote specialization. Tokens are in ovals whereas classes are in rectangular boxes. The TELL commands for some of these objects are as follows:

Example 3.4:

```
IndividualClass Department IN M2_Class
WITH
    necessary
            deptName :String
    attribute
            staff    : Employee;
            nonStaff : Others
END Department
```

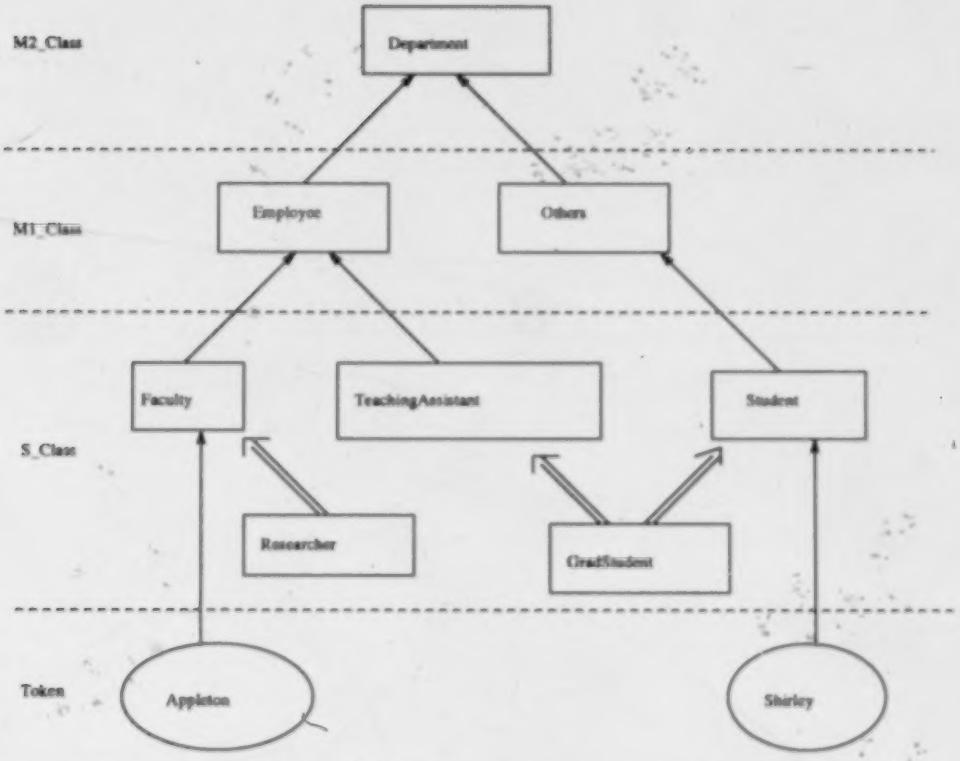


Figure 3.1: Object Hierarchies of a Department Information System

Example 3.5:

```
IndividualClass Employee IN M1_Class, Department
WITH
    necessary, single
            employeeId  : Integer
    attribute
            name        : String;
            salary      : 0..80000;
            hiringDate  : Date
END Employee
```

Example 3.6:

```
IndividualClass Faculty IN S_Class, Employee
WITH
    necessary, single
            facId       : Integer
    attribute
            name        : String;
            degree      : DegreeType;
            joinDate    : Date
END Faculty
```

Example 3.7:

```
IndividualClass Researcher IN S_Class, Employee
IsA Faculty
WITH
     paper      : Publication;
     conference : Conference
END Researcher
```

Example 3.8:

```
Individual Appleton IN Faculty, Token
WITH
    necessary, single
            facId       : 193384
    attribute
            name        : Larry Appleton;
            degree      : Ph. D.;
            joinDate    : 1983/9/1
END Appleton
```

The previous Telos description specifies that every **Department** has a name which is mandatory, and attributes **staff** and **nonstaff**. **Department** has two classes of objects: **Employee** and **Others**. **Employees** are **Faculty** members and **Teaching Assistants** while

Others are Students. Researchers are specialization of Faculty and have two attributes, paper and conference in addition to the attributes of Faculty. Finally, Appleton is a token that describes a particular faculty member.

Student and Shirley were defined in Example 3.2 and 3.3.

Telos supports multiple inheritance. The following object definitions illustrate that:

Example 3.9:

```
IndividualClass TeachingAssistant IN S_Class, Employee
WITH
    attribute
            supervisor      : Employee;
            academicAdvisor : Faculty;
            assignment      : Assignment
END TeachingAssistant
```

Example 3.10:

```
IndividualClass GradStudent IN S_Class, Employee, Others
IsA Student, TeachingAssistant
END GradStudent
```

Example 3.11:

```
Individual Yu IN GradStudent, Token
WITH
   necessary, single
     id               : 197652;
   attribute
     name             : Xia Yu;
     credits          : 15;
     academicLevel    : MS;
     supervisor       : Paul Sorenson;
     academicAdvisor  : Paul Sorenson;
     assignment       : CMPUT 401
END Yu
```

Now, in the above it is not clear whether the attribute category academicAdvisor comes from TeachingAssistant or Student. Therefore, the FROM clause is used instead as follows:

```
academicAdvisor FROM TeachingAssistant

                : Paul Sorenson;
```

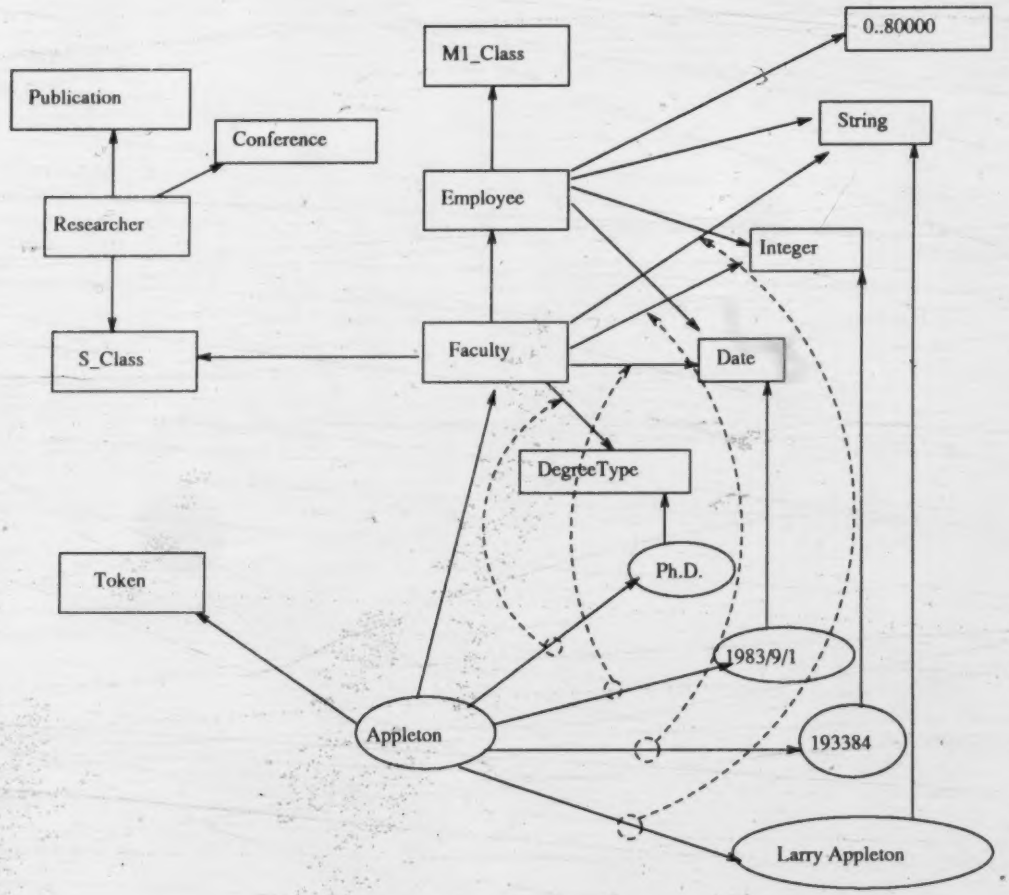


Figure 3.2: Semantic Net Representation of Department Information System

The semantic net representation of Department Information System has been given in Figure 3.2. For each object, its relationships to other objects and its attribute types are shown by solid arcs. The entities in oval represents ground instance or data. The dotted arcs represents the relationships between the attributes of the object definition and actual data.

3.4 Representation of Temporal Knowledge

The treatment of time in Telos is based on Allen's temporal framework [Allen, 1981, 1983, 1984] [Allen & Hayes, 1985] but is modified in three ways [Mylopoulos et al., 1990]:

1. Allen's original seven temporal relationships is extended to the relations shown in Figure 3.3. Additionally, it includes **at** and **meets** temporal primitives.

2. It has incorporated temporal constants such as:

   - time constants for year, month, day, hour, minute and second intervals, e.g., 1990/11/8 means November 8, 1990.
   - time constants of durations, e.g., (1989/12 .. 1990/11); left-infinite durations, e.g., (* .. 1988/3) and right-infinite duration, e.g., (1988/3 .. *), and
   - special constants **Alltime** denoting infinite time interval and **Now** denoting system's current time.

3. The power of temporal assertions has been restricted, e.g., only a single temporal relation is allowed to appear in any temporal component of a definition.

Telos keeps the *history* of the domain as well as the *system's belief* about the history. This feature enables it to represent state changes of the domain and allows historical queries with respect to the system's current state as well as a previous state.

Page 32 (Figure 3.3) has been taken out for copyright restrictions.
This figure shows temporal relations with their inverses.
It is available in [Koubarakis et al., 1989a] (see bibliography)

History time interval is provided by the user while belief time interval is generated by the system. The system keeps *believing* something until the belief is revised by UNTELL or RETELL commands.

The class identification clause, IN clause, IsA clause and each attribute of a WITH clause have optional temporal constraints associated with them. In absence of the temporal constraint the default (at systime .. *) is assumed which is the system's execution time of that command.

We give an example below to show temporal constraints:

Example 3.12:

```
Individual CompScChair IN ChairPerson (at 1988/9..*)
WITH
    attribute
        deptChair    : Paul Sorenson (at 1989/6 .. *)
    gradChair
        currentChair : Tony Marsland (during 1990/5);
                     : Tamer Ozsu    (before 1990/5/1)
END CompScChair
```

The example above says that CompScChair is an instance of the class ChairPerson from September, 1988 onward. Paul Sorenson is the deptChair from July, 1989 onward. Tony Marsland was the gradChair in May, 1990 and Tamer Ozsu was the gradChair sometime before May 1, 1990, but does not say precisely when. This also illustrates incomplete temporal information can be represented in Telos.

3.5 Assertions in Telos

Telos has incorporated a typed first order language with equality in order to specify *integrity constraints* and *deductive rules* as well-formed formulae. Integrity constraints are used in consistency checking and deductive rules are used in answering queries.

The following predicate symbols have been defined in Telos [Koubarakis et al., 1989a]:

1. prop(p,x,y,z,t) means

   the proposition p has components x, y, z and t.

2. instanceOf(x,y,t1,t2) means

   x is an instance of y for the time period t1. This is believed by the system for time period t2. In infix notation, instanceOf(x,y,t1,t2) $\longleftrightarrow$ x in y [at t1 - believed at t2].

3. att(x,y,t1,t2) means

   x has an attribute value y with attribute category att for the time period t1. It is believed by the system for the time period t2.

4. isa(x,y,t1,t2) means

   x is a specialization of y for the time period t1. It is believed by the system for the time period t2.

5. instanceOf(x,y,t) means

   x is an instance of y for the time period t.

6. att(x,y,t) means

   x has an attribute value y with attribute category att for the time period t.

Telos offers the following functions [Mylopoulos et al., 1990]:

- The dot function x.l[r t]

  returns the attribute values of proposition x which are in attribute category l during an interval which is in relation r with time t.

- The hat function x^l[r t]

  returns the attribute values of proposition x which are in attribute label l during an interval which is in relation r with time t.

- The bar function x|l[r t]

  returns all attribute propositions with source x, which are in attribute categor l during an interval which is in relation r with time t.

- The exclamation mark function x!l[r t]

  returns all attribute propositions with source x, which are in attribute label l during an interval which is in relation r with time t.

The temporal constraints in these functions are optional. A special variable **this** has been defined in which ϕ(this) denotes (Forall x/C ϕ(x)) where C is the class for which ϕ(this) has been defined.

Each integrity constraint and deductive rule has optional temporal constraints associated with them. But for integrity constraints these temporal constraints must be of the form,

```
(at <time interval>)
```

The deductive rules are of the form:

```
(Forall x1/s1) ... (Forall xn/sn) W ==> A
```

where W is a conjunctive formula and A is a positive atomic formula with a 3-place predicate as defined earlier.

Example 3.13:

```
IndividualClass ChairPerson IN Employee
WITH
    attribute
         deptChair : Employee;
         gradChair : Employee
    integrityConstraint
            :$(Forall d/Department) (Forall x/Chairperson)
              (Forall t1,t2/TimeInterval)
      (deptChair(d,x,t1,t2) ==> (x MemberOf d.staff))$ (at 1990..*)

END ChairPerson
```

The integrity constraint in the example above states that from 1990 onward the deptChair of a department must be a member of the staff (i.e. can not be nonstaff). Here, MemberOf is a built-in set relation.

Example 3.14:

```
IndividualClass Student IN S_Class
WITH
    necessary, single
                id              : Integer
    attribute
                name            : String;
                credits         : Integer;
                academicAdvisor: Faculty;
                academicLevel   : Year
    deductiveRule
       :$(Forall x/Student) (Forall y/Integer)
        (Forall t1,t2,t3/TimeInterval)
                (credits(x,y,t1,t2) and y>90  ==>
                 academicLevel(x,Senior,t3))$
END Student
```

The deductive rule above says that if a student earns more than 90 credits (s)he is classified as a Senior.

3.6 Meta-Definitions

One of the novel features of Telos is that it allows users to define their attribute classes with its meta-attribute definition facility.

The attribute category **single** in Student means the value of its attributes must be one. This semantic restriction is encoded in Telos in the following manner (the example is similar to the one in [Mylopoulos, et al., 1990]):

Example 3.15:

```
TELL CLASS Single
COMPONENTS [Class, single, Class]
IN  AttributeClass, M1_Class
WITH
    integrityConstraint
        :$ (Forall u/Single) (Forall p,q/Proposition)
           (p in u and q in u and from(p)=from(q) and
            when(p) overlaps when(q) ==> p=q)$
END Single
```

This facility also allows to state the nature of an assertion. For example, if we want an assertion to be a precondition to a proposition we can include it under attribute category **precondition** and define it as below [Mylopoulos et al., 1990]:

Example 3.16:

```
TELL CLASS PreCondition
COMPONENTS [Class, precondition, Assertion]
IN AttributeClass, M1_Class
WITH
    integrityConstraint
        :$ (Forall p/PreCondition) (Forall q/Proposition)
           (Forall t/TimeInterval)
           (q in from(p) [at t] ==> (Exists t1/TimeInterval)
           (t1 overlaps t) and Holds(to(p)))$
END PreCondition
```

3.7 Other Features

Telos also offers querying and update facilities. Queries are specified by a RETRIEVE command that answers them from the *definitional part* of the knowledge base or by an ASK command that answers them from the *assertional part*.

The UNTELL command makes the system *cease to believe* about an object UNTELLed but does not delete it from the knowledge base. The command RETELL allows revising information.

At this point, our system does not take into account these operations; therefore, we avoid describing them here. Please refer to [Koubarakis et al., 1989a] and [Mylopoulos et al., 1990] for details.

Chapter 4

Sentence Generation

In this chapter we discuss how sentences are generated from any Telos object definition. Our system, known as the Generator of English Text from Specifications (GETS – pronounced *jets*), processes the whole specification object by object. Depending on the requirements of the user, it chooses relevant objects to describe. The content selection strategy will be discussed in Chapter 5. In this chapter, we focus on the treatment of a single object only.

4.1 Sentence Generation Mechanism

For the purpose of this thesis, a Telos object description is partitioned into three major components:

a) *Introductory Component*: This component includes type, IN and IsA clauses, and denotes the type of an object and its classification and specialization information.

b) *Attribute Component*: This component follows a WITH clause and contains a set of attributes. Attributes are grouped together under attribute categories.

c) *Assertional Component*: It includes integrity constraints and deductive rules which are first-order formulae. Assertions, too, follow a WITH clause.

In the rest of the chapter we show how sentences are generated from the above components. We begin with a functional decomposition of GETS. The sentence generation system of GETS requires the following components:

a) Recognizer

b) Arranger,

c) Preliminary Sentence Generator (PSG),

d) Local Refiner, and

e) Global Refiner.

Figure 4.1 shows these components with their inter-relationships. The square boxes represent the main components of the sentence generator of GETS, the ovals are input rule knowledge bases, and the circles are repositories of other information, such as tags.

The input to the *recognizer* is the whole object definition. The recognizer is essentially a parser whose output is the three identified components of the object definition described above, i.e., introductory, attribute and assertional components.

The input to the *arranger* is the attribute and assertional components of the object. The task of the arranger is to move assertions to the end of the object definition, and to re-order the attributes on the basis of attribute categories. The re-ordering is done in order to improve the organization of the text. Additionally, the arranger groups the attributes of same types under each category together, and generates *tags* to denote the appearance of a category label at multiple places. These tags are used later by the global refiner.

The *preliminary sentence generator* (PSG) has as input the three major object components just described and generates, as output, initial sentences that may later be revised. The introductory and attribute components are translated into preliminary sentences with the help of a phrase structured grammar. Assertions are sent to the *assertion handler* where they are translated into sentences. The translation process is described in Section 4.5. Before an attribute component is processed attributes under each category are partitioned. The attributes in each partition become the content of one sentence. At each partition point a *tag* is generated that is later used by the *global refiner*.

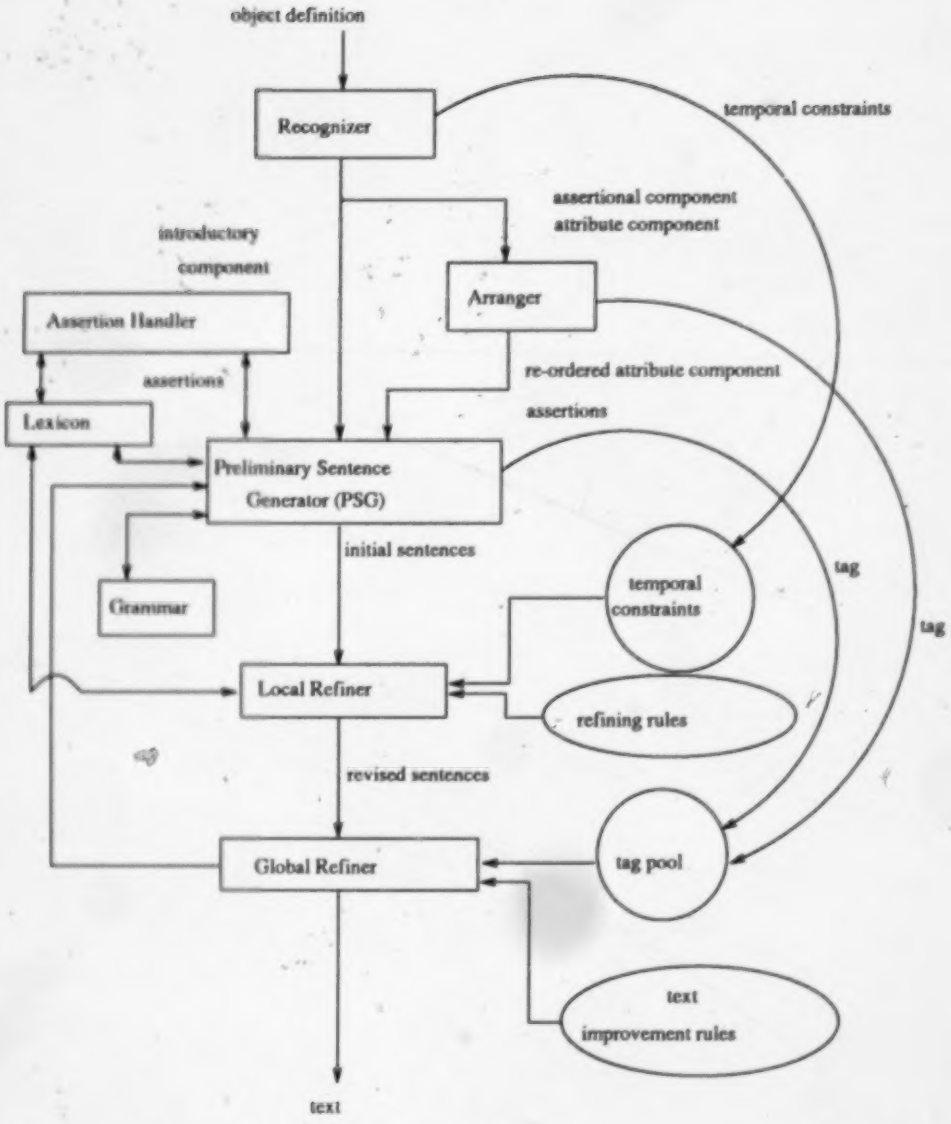


Figure 4.1: Components of the Sentence Generator of GETS

The output of the PSG is sent, as input, to the *local refiner*. The tasks of the local refiner are proper capitalization, ensuring number and subject-verb agreement, choosing articles, case-forms, gender specific pronouns and substituting phrases, splitting compound words and punctuation. The local refiner also integrates phrases of the temporal constraints into the sentence.

Once all the initial sentences have been locally revised, they are sent to the *global refiner* which ensures coherence among sentences, reduces redundant phrases and maintains connectivity among the three major components of the object. Its objective is to enhance the overall readability of the text. The global refiner may re-submit a sentence to the PSG if the global refiner considers its structure unacceptable.

Let us now discuss what motivated this functional decomposition of the sentence generation system. The recognizer is necessary because a Telos description must be syntactically analyzed. The arranger has been added to *optimize* the attribute order to give a better organization of generated text. The PSG generates only initial sentences that are later revised, with the help of a context-free grammar. This approach is used to keep the grammar size small. Incorporating details (e.g., alternate number agreement) often results in an explosion in the number of production rules. These details are added in the later stage by the local refiner. Finally, at the individual sentence level it is difficult to see how sentences co-relate with each other; therefore, a global refiner has been added. One of the criticisms of many systems is that they ignore text improvement [Mann, 1983a]. The local and global refiners in our system are used to enhance the readability of the text produced.

4.2 Identification Sentences

The general form of the introductory component of an object definition is :

<IndividualType> <IndividualIdentifier> IN <class> {"," <class>} [IsA <class> {"," <class>}] Here, [x] denotes 0 or 1 appearances of x, a syntactic form, and {x} denotes 0 or more.

The IN and IsA clauses in a Telos specification refer to the classes to which an object belongs. GETS generates a single sentence containing these identification clauses. For clients (system users), the built-in class types such as S_Class, M1_Class etc. are ignored. For example:

```
IndividualClass Student IN S_Class
```

is translated into:

There is a class Student.

But the following:

```
IndividualClass Employee IN M1_Class, Department
```

is translated into:

The class Employee is a member of the class Department.

The M1_Class is ignored in this case. Also note the difference with the previous sentence. The latter begins with a definite article *the*, whereas the former starts with *There is* ..., because there is no object to support it. The rule is that if the object list following the IN clause is empty or contains only built-in classes then the sentence structure begins *There is* ....

If multiple classes follow the IN clause, they are joined by the connective, *and*. For example,

```
IndividualClass GradStudent IN S_Class, Employee, Others
```

generates,

The class GradStudent is a member of the classes Employee and Others.

The correct pluralization of the word class is done by the local refiner.

If a class definition includes an IsA clause, it is taken into account in the sentence in the following way:

```
IndividualClass Researcher IN S_Class, Employee
IsA Faculty
```

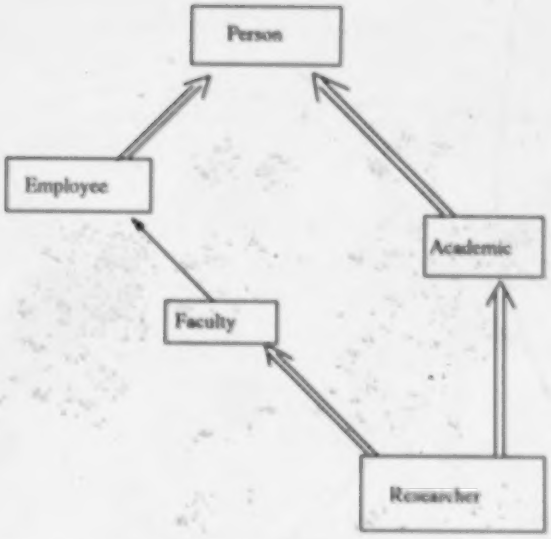


Figure 4.2: Hierarchies for Employee, Researcher and Faculty

The above definition is translated into:

The class Researcher is a member of Employee and is a special Faculty.

The word **special** has been chosen from the fact that **Faculty** is a specialization.

This can also be translated into:

A researcher is an employee and a faculty. or

A researcher is an employee as well as a faculty.

We assume that the elements of an object class are known by the same name, only in lower cases. For example, members of the class Researcher are known as researcher. In the above sentence, the local refiner chose small lettered **researcher** with an indefinite article **a**, but in the previous sentence a definite article was chosen because it was referring to the class Researcher. The substitution of the phrase **as well as** is also the job of the local refiner. It could also generate:

Researchers are employees as well as faculty.

When the second style is taken, a check is made of the type of objects.

Following the same generation style, the definition:

```
IndividualClass Employee IN M1_Class, Department
```

obviously could not be translated into:

An employee is a department.

The reason is that in the former case Researcher, Employee and Faculty have the same type. This type-matching information can be provided by the specifier, or can be deduced from the object relation tree. For example, from Figure 4.2 it can be deduced that Employee, Researcher and Faculty have the same type Person. But only specialization hierarchies (denoted by double arrows) and not class hierarchies help in deduction.

GETS also knows when to skip an IN clauses, as in:

```
IndividualClass Researcher IN S_Class
IsA Faculty
```

This simply generates:

The class Researcher is a special Faculty. or

A researcher is a faculty.

The second type of sentence is generated when the objects are type-compatible.

For ground instances or tokens the sentence begins with the defined object as the subject of the sentence. The declaration

```
Individual Shirley IN Student, Token
```

produces,

Shirley is a student.

Ground instances are identified with the keywords *Individual* and *Token*.

4.2.1 Examples of Generation from Introductory Component

By using the following production rules of a grammar we are able to generate some sentences from the introductory component.

1. <sentence> → <subj> <verb> <obj>

2. <subj> → <det> <IndividualIdentifier>

3. <det> → A @ <*IndividualIdentifier*> *is type compatible with* <*class*>
   | The <key1> @ *otherwise*

4. <verb> → <aux>

5. <aux> → is

6. <obj> → a <class> @ <*IndividualIdentifier*> *is type compatible with* <*class*>
   | <key2> <class>

7. <key1> → class

8. <key2> → a member of the <key1>

The @ character above denotes that the corresponding production is taken only if the condition holds.

4.2.1.1 Example 4.1

Let us assume that we are given a part of a definition as follows:

IndividualClass Researcher IN Employee

The output of the recognizer for this definition fragment, corresponding to the structure given at the beginning of Section 4.2, is as follows:

```
<IndividualType>      --> IndividualClass
<IndividualIdentifier> --> Researcher
<class>                --> Employee
```

The arranger does nothing since only the introductory component is being processed.

Since Researcher is type compatible with Employee, PSG, with the help of the generative grammar, produces this sentence:

A Researcher is a Employee

The trace of the above sentence generation is as follows:

```
<sentence> --> <subj> <verb> <obj>  (by production 1)
    --> <det> <IndividualIdentifier> <verb> <obj>  (by production 2)
    --> A Researcher  <verb> <obj>
                        (by production 3, and output of the recognizer)
    --> A Researcher  <aux> <obj>  (by production 4)
    --> A Researcher  is  <obj>  (by production 5)
    --> A Researcher  is  a Employee
                        (by production 6, and output of the recognizer)
```

The local refiner corrects the article in front of Employee according to an article rule that states that, unless it is in the exception list, a word beginning with a vowel requires the article an, and puts Researcher and Employee in lower cases. Therefore, the revised sentence is:

A researcher is an employee.

4.2.1.2 Example 4.2

Assume that the following partial definition is being processed:

IndividualClass Employee IN M1_Class, Department, Administration

The output of the recognizer is:

```
<IndividualType>       --> IndividualClass
<IndividualIdentifier> --> Employee
<class>                --> (M1_Class, Department, Administration)
```

The value of <class> is a list with 3 elements. PSG generates :

The class Employee is a member of the class (M1_Class, Department, Administration)

Now, the local refiner identifies M1_Class as a built-in class and therefore, deletes it. It also pluralizes the word class and adds a conjunction connector. The revised sentence is:

The class Employee is a member of the classes Department and Administration.

The grammar given above is limited. Although it can generate the sentences of Example 4.1 and 4.2 it can not generate the other sentences that we have seen in this section.

4.3 Attribute Descriptions

In this section we discuss generation of sentences from attributes following the WITH clause that are not assertions. The attribute component of an object definition has the form:

WITH <AttributeDeclaration> {<AttributeDeclaration>}

where <AttributeDeclaration> has the form,

<AttributeCategory> {"," <AttributeCategory>} <attribute> {"," <attribute>}

and <attribute> is of the form,

[<AttributeLabel>] ":" <AttributeValue>

4.3.1 Abstract Objects

The *arranger* re-orders the attributes of non-token objects such that categories of the same labels appear together. It tries to put a maximum number of identical categories together. Suppose, we have a set of attribute categories as in the following:

```
... ... ...
WITH
     necessary
               ... ...
     input, single
               ... ...
     control
               ... ...
     output, single
               ... ...
     control, single
               ... ...
     necessary, single
               ... ...
END
```

The first goal is to group together all *single* categories creating the following order:

```
... ... ...
WITH
     necessary
               ... ...
     input, single
               ... ...
     output, single
               ... ...
     control, single
               ... ...
     necessary, single
               ... ...
     control
               ... ...
END
```

Now, we move *necessary* right after *necessary, single*, and give up the maximal adjacency rule to insert *control* right after *control, single*. Thus the final order becomes:

```
... ... ...
WITH
     input, single
               ... ...
     output, single
               ... ...
     control, single
               ... ...
     control
               ... ...
     necessary, single
               ... ...
     necessary
               ... ...
END
```

The other tasks of the arranger is to group attributes having the same type under a category, and to generate tags to mark which category labels occur together.

The *PSG* determines the contents of a sentence. Several heuristics are applied for this purpose. The *cardinality* heuristic determines which attributes go into a sentence. The heuristic is that if there are n attributes in one category then there are $n/3$ partitions. But if the last partition has only one attribute it is moved to the previous partition. For example, if an attribute category contains 6 attributes they may be split into two sentences each having 3 attributes. If it has 7, they may be split into partitions of size 3 and 4 attributes.

We will describe the attributes of the object **Student** below:

Example 4.3

```
IndividualClass Student IN S_Class
WITH
    necessary, single
            id              : Integer
    attribute
            name            : String;
            credits         : Integer;
            academicAdviser : Faculty;
            academicLevel   : Year
END
```

For each attribute category one or more sentences is generated. That is, in the current example one sentence will be generated from the attributes under the category *necessary, single* and one or more from category *attribute*. Note that *attribute* is a built-in category when no other user-defined category exists.

In Example 4.3 the sentences generated are:

A student has an id (which is an Integer) which is necessary and unique. (S)he also has credits (which is an Integer), a name (which is a String), an academicAdvisor who is a faculty and an academicLevel (which is a Year).

The sentence fragments appearing in parenthesis are the parts that are later discarded by the *local refiner*. This is because these attribute types are *trivial*, and they can be ignored. GETS identifies two kinds of trivial types:

a) built-in types: e.g., Integer, String etc. and

b) some user-defined types that are *commonly known*, e.g., Date, Year etc. Commonly known types must be explicitly identified by the user.

There are other cases when the which-clause can be disregarded. These *redundancy removal* cases occur when there are identical attribute labels and types or compound words having part of the word as the type name (upper/lower cases ignored). For example, consider the following example:

```
attribute
        name          : Name;
        orgCommittee : Committee;
```

These attributes are translated into:

It has attributes name and orgCommittee.

as opposed to:

It has attributes name which is a Name and orgCommittee which is a Committee.

The sentence connective **also** has been used in the second sentence in the translation of Example 4.3 in order to offer continuity. This is done by the *global refiner*.

In Example 4.3 we need to identify that **Student** is singular. We also need to know article, number, case-form and gender of each attribute. With this information, the local refiner can determine that, for example, **credits** requires no article in front of it and **academicLevel** and **academicAdvisor** require the article *an* while **name** requires the article *a*.

The local refiner has a set of defaults and a set of rules with exceptions to accomplish article, number and gender agreements. For example, the article *a* would be used as default unless it satisfies an exception or the rule that words starting with a vowel require an *an*. Exceptions take care of cases such as 'university' requires the article *a* even though its first letter is a vowel, but 'hour' requires the article *an*.

For determining number agreement, the same default and rules technique can be applied. One rule is that words ending with **s** or **ies** are plural with the possible exceptions such as 'mathematics' which is singular, and 'criteria' which is plural. These rules conform with standard English grammar. Only words appearing in the

specification will be allowed in the exception table, thus making the table finite in size.

In GETS, gender is specified as one of male, female or neuter that we will denote as G0, G1, and G2 respectively. From this information some of the case forms and pronouns can be determined. For example, G0 implies the form *his*, G1 implies *her* and G2 implies *its*. Again, either G0 or G1 implies the use of *who* while G2 implies *which*.

An alternate set of sentences corresponding to the above mentioned example is:

The class Student has a necessary and unique attribute id. The other attributes are credits, name, academicAdvisor and academicLevel.

GETS has the flexibility of generating alternate sentences. In the first sentence of the last example if id had a type idNumber the following would be generated:

A student has an id which is an idNumber which is necessary and unique.

This sentence is less readable than the sentence:

The class Student has a necessary and unique attribute id which is an idNumber.

The local refiner also substitutes unique for *single* which has been gleaned from the lexicon. It can also split compound words, e.g., academicLevel to academic level.

The global refiner combines sentences to 'bridge' between identification sentences and attribute descriptions. Returning to Example 4.3, after putting together the identification sentence and attribute desciption, we have:

There is a class Student. The class Student has a necessary and unique attribute id. The other attributes are credits, name, academicAdvisor and academicLevel.

These can be combined into:

There is a class Student that has a necessary and unique attribute id. The other attributes are credits, name, academicAdvisor and academicLevel.

The general rule is that if the object of a sentence (Student) is the subject of the next sentence then they may be combined into one.

4.3.2 Example of Generation from Attribute Component

In this subsection, we show how sentences are generated from the attribute component of an object definition, and illustrate how the components of the sentence generator work. Let us consider the following object definition which is similar to Figure 4 in [Mylopoulos et al., 1990]:

Example 4.4

```
IndividualClass ConferenceOrganization IN ActivityClass
WITH
     input, single
            budg          : Budget
     control, single
            nm            : Name;
            endDate       : Date;
            orgCommittee: Committee;
            startDate     : Date
     deductiveRule:
            $ ... ... ... $
     control
            sponsor       : Organization
     output, single
            conf          : Conference;
            finalReport   : Report
     integrityConstraint:
            $ ... ... ... $
       ...      ...      ...      ...
END
```

Now, we will show in a step-by-step fashion how text is generated from this definition. After the actions of the recognizer, the arranger moves deductiveRule and integrityConstraint to the bottom, arranges the attributes to give optimal adjacency of the similar category labels, groups the attributes having type 'Date' together, and generates a tag at the third occurrence of the category label 'single'. At the conclusion of the arranger's actions, the attribute component is as below:

```
IndividualClass ConferenceOrganization IN ActivityClass
WITH
     input, single
            budg          : Budget
     control, single
            nm            : Name;
            orgCommittee: Committee;
            startDate, enddate    : Date
     output, single
            conf          : Conference;
```

```
                    finalReport : Report
tag ("single" occurred 3 consecutive) ------------->
              control
                    sponsor     : Organization
              deductiveRule:
                    $ ... ... ... $
              integrityConstraint:
                    $ ... ... ... $
                ...     ...     ...     ...
        END
```

Now, PSG decides to generate one sentence from each category and therefore generates tags at the end of each category. The deductiveRules and integrityConstraint are sent to the Assertion Handler where they are realized as sentences. With the help of a grammar, the PSG generates the following preliminary sentences.

(1)The class ConferenceOrganization is a member of the class ActivityClass.

(2) It has a unique input attribute budg which is a Budget.

(3) It has unique output attributes conf which is a Conference and finalReport which is a Report.

(4) It has unique control attributes nm which is a Name, orgCommittee which is a Committee, (startDate, endDate) which is a Date.

(5) It has a control attribute sponsor which is an Organization.

The local refiner performs redundancy removal in sentence 3 on finalReport and rearranges sub-sentences to remove ambiguity. The rearrangement rule is that, in a sentence, attributes without a which-clause precedes one with a which-clause in order to reduce ambiguity. The revised sentence 3 is:

(3′) It has unique output attributes finalReport, and conf which is a Conference.

In sentence 4, the local refiner identifies Date as a *trivial type* and performs *redundancy removal* on orgCommittee. The revised sentence 4 is:

(4′) It has unique control attributes orgCommittee, startDate, endDate and nm which is a Name.

The global refiner knows from a tag that *single* appears three times and reduces redundancy by adding a sentence after the 4th sentence that states this property. In

general, a sentence is added at the tag point which states the tag's property. It also introduces sentence connectives, e.g., Additionally and one additional, and replaces the pronoun in sentence 5 with the original subject.

Taking all these into consideration, the final form of the paragraph is as below:

The class ConferenceOrganization is a member of the class ActivityClass. It has an input attribute budg which is a Budget. It has output attributes finalReport, and conf which is a Conference. Additionally, it has control attributes nm which is a Name, orgCommittee, startDate and endDate. All these attributes must be unique. ConferenceOrganization has one additional control attribute, sponsor, which is an Organization.

The general rule of using sentence connectives, e.g., also, additionally, one additional etc., is that they are used in subsequent descriptions of attributes. Additionally is used at the beginning of the sentences, also is used right after the subject (as in: It also has ...) and one additional is used after auxilary verbs (e.g., It has one additional attribute ...).

The pronominalization rule is that usually the first sentence in an object description starts with the object name as the subject and for subsequent reference a pronoun is used. But after a number of consecutive sentences the original subject is used again.

4.3.3 Tokens

Up to now, we have dealt with attributes and their labels and types but did not consider actual data. In the description of a specification some token may be translated as illustrations of abstract objects. GETS treats token descriptions slightly differently than abstract objects. For tokens, the attribute category labels normally do not participate in the sentence. For example, consider the description of Shirley, which is a ground instance of the class Student, given below:

Example 4.5

```
Individual Shirley IN Student, Token
WITH
```

```
necessary, single
            id              : 904381
attribute
            name            : Shirley Bates;
            credits         : 102;
            academicAdviser : Paul Sorenson;
            academicLevel   : Senior
END
```

The generated sentences for this token-level object are:

Shirley is a student. Her id is 904381, credits are 102 and name is Shirley Bates. Her academicAdvisor is Paul Sorenson and academicLevel is Senior.

With the additional knowledge that one of the attributes (name) is an *elaboration* of the token name (Shirley) and one of the attributes (academicLevel) is a qualifier (adjective, in grammatical sense) for the object name (Student), the elaboration attribute is used as the subject and the qualifier as an adjective to the object of the first sentence. This additional knowledge must be provided by the specifier.

Using the above rule, GETS revises those sentences to:

Shirley Bates is a Senior student. Her id is 904381 and credits are 102. Paul Sorenson is her academic advisor.

In the last sentence, note the subject-object switch and splitting of the compound word academicAdvisor. GETS generates different kinds of sentences to reduce monotonicity.

For the generator to work properly the attribute labels need to be nouns or compound words with noun sense and the attribute category labels are to be adjectives.

In the absence of attribute labels, attribute category participates in the description of a token. As an example, consider the following attributes:

```
members
    chairPerson : Kathy Evans;
                : Tom Kullar;
                : Ed Carpenter;
```

This is translated into:

The chairPerson is Kathy Evans. The other members are Tom Kullar and Ed Carpenter.

Here the attribute category label members has been used to describe the last two attributes since no labels exists for them.

Lastly, the FROM clause is ignored but taken into consideration for non-tokens.

4.4 Incorporating Temporal Expressions

In a schema of the TELL command below we show all possible places where temporal expressions can appear in a definition. The temporal components are optional.

```
<token-or-class-def> <id> (<temporal constraint>)
IN <class> (<temporal constraint>)
IsA <class> (<temporal constraint>)
WITH
    <category>
        <label> : <value> (<temporal constraint>)
END
```

Each <temporal constraint> is of the form <time relation> <time constant>. Among the temporal relations of Figure 3.3, the four that can appear in <time relation> are during, at, before, and after.

These temporal relations coupled with various temporal constants are translated as shown in Table 4.1. The specific temporal constants may be replaced with any temporal constant in that table.

Table 4.1: Translations of Temporal Constraints.

const./temp. rel.	at	before	after	during
1990/11	at November, 1990	before November, 1990	after November, 1990	during November, 1990
1989/5.. 1990/6	between May, 1989 and June, 1990	×	×	between May, 1989 and June, 1990
*..1983/3	until March, 1983	before March, 1983	×	until March, 1983
1983/..*	from March, 1983 onward	×	from March, 1983 onward	from March, 1983 onward
Alltime	for alltime	×	×	for alltime
Now	at system's current time	before system's current time	after system's current time	during system's current time

Once the phrase corresponding to a temporal constraint is determined, it is added at the end of the sentence generated without the temporal component. Example 3.11 is translated into the following paragraph without considering temporal constraints:

CompScChair is a ChairPerson. Its deptChair is Paul Sorenson and currentChair is Tony Marsland. Another gradChair is Tamer Ozsu.

With temporal constraints taken into consideration, and assuming that the generation was executed in May, 1990 the paragraph becomes:

CompScChair is a ChairPerson for September, 1988 onward. Its deptChair is Paul Sorenson from June, 1989 onward and currentChair is Tony Marsland during May, 1990. Another gradChair was Tamer Ozsu before May 1, 1990.

In the last sentence, the auxilary verb is has been changed to was. As a general rule, when incorporating a temporal constraint it is compared with system's current time and an appropriately tensed verb is chosen. Normally, GETS generates sentences in present tense form.

4.5 Sentences from Assertions

The temporal notions, especially the concepts of history and belief time in Telos, are largely for the sake of temporal reasoning. Therefore, GETS ignores temporal

variables and relations while generating sentences from assertions. In Example 4.6 that follows, t1 is the time for which the predicate **address** holds, and t2 is the time for which the predicates **firstAuthor** and **replyAddress** hold. And, t is the system's belief time of **address** and **firstAuthor**. Now, if a temporal query is issued, these variables play an important role in answering it; but, as we show later, the assertion is translated into:

The address of the first author of a conference paper is the reply address of that conference paper.

This sentence is independent of any temporal reference. We recognize that it may not be the case in all assertions but taking care of temporal variables poses an immense problem and we leave this for future research.

Even though integrity constraints and deductive rules are used for different purposes in Telos, they can be translated into natural language by employing the same strategy because they are similar.

GETS's assertional sentence generation consists of three steps:

1) identifying definitions,

2) applying reduction methods and generating sub-sentences, and

3) combining sub-sentences and performing overall refinements.

In step 1, GETS identifies which predicates are *definitional*. The definition of x is determined by the first predicate in which x appears as the second argument. For example, **author(y,x,t1,t2)** is the definition of x as an author.

In step 2, sub-sentences are generated corresponding to each predicate. The predicate **att(x,y,t1,t2)** is translated into:

y is the att of x

and **instanceOf(x,y,t1,t2)** is translated into:

y is a member of x or x is a y.

Reduction rules include transitivity (e.g., x is y and y is z implies x is z), placement of not (e.g., **not (Exist t/TimeInterval) member(x,y,t)** implies **(Forall t/TimeInterval)**

not member(x,y,t)), and or-inclusion (e.g., (chairperson(CompSc,x,t1,t2) or chairperson(CompEngg,x,t1,t2)) implies chairperson((CompSc or CompEngg),x,t1,t2)).

In step 3, the whole sentence is generated from the sentence-parts. An implicative assertion (==>) is translated into an "If ... then ..." structure. If a variable for which a definition predicate exists occurs more than once it is *dropped*, i.e., deleted. But after dropping definitional predicates and temporal relations, if the antecedent becomes empty then the if-then form is not generated. The refinement makes appropriate pronominalization, choice of articles, numbers and case forms and eliminates redundancy.

The following examples, taken from [Topaloglou and Koubarakis, 1989], illustrate the process.

```
Example 4.6
 (Forall x/ConferencePaper, y/Author, z/Address, t,t1,t2/TimeInterval)
 (address(y,z,t1,t) and firstAuthor(x,y,t2,t) and t2 during t1
      ==> replyAddress(x,z,t2) )
```

The predicate address is the definition of z (replyAddress is not, since address occurs first) and firstAuthor is the definition of y. These are translated, respectively, as:

z is the address of y

y is the firstAuthor of x

By transitivity and noting that x has a type ConferencePaper, the following can be produced:

z is the address of the firstAuthor of a ConferencePaper x.

The predicate replyAddress produces:

z is the address of x.

Now, after dropping the predicate address and firstAuthor (because they are definitional) and dropping temporal relation t2 during t1, the antecedent of the assertion becomes null. Therefore, the if-then form is not generated. GETS just replaces the

definition of z in the sentence corresponding to replyAddress. Thus the final sentence is:

The address of the firstAuthor of a ConferencePaper is the replyAddress of that ConferencePaper.

At refinement stage, the pronoun *that* has been chosen in the second reference since ConferencePaper has already been mentioned. The local refiner also normalizes compound words ConferencePaper, firstAuthor and replyAddress:

The address of the first author of a conference paper is the reply address of that conference paper.

The following example has a disjunctive clause in it:

```
Example 4.7
 (Forall x/EspritProject) (Forall y/Employee)
 (Exists t1,t2/TimeInterval) (Exists t3,t4/TimeInterval)
 (projStaff(x,y,t1,t2) ==>
    (instanceOf(y,Researcher,t3,t4) or instanceOf(y,Consultant,t3,t4))
    and (t1 during t3) and (t2 during t4)
```

Here, projStaff is definitional for y, making the antecedent null. By reduction, instanceOf(y,(Researcher or Consultant) t3,t4) is produced.

The predicate projStaff generates,

y is the projStaff of an EspritProject x.

The predicate instanceOf generates,

y is a Researcher or a Consultant.

Substituting the definition of y in the second sentence, the natural language description of the assertion is:

The projStaff of an EspritProject is a Researcher or a Consultant.

The following example shows how negation is handled:

Example 4.8

```
(Forall x/ConfPaper, y/Author, t1,t2/TimeInterval)
(author(x,y,t1,t2) ==>
  not (Exists t/TimeInterval) referee(x,y,t,t2) )
```

In the above example, author is the definition of y and generates,

y is the author of a ConfPaper x.

The reduction mechanism places *not* right before referee and generates,

y is not the referee of a ConfPaper x.

Now, replacing the definition of y from the first sentence in the second sentence, and refining:

The author of a confpaper must not be the referee of it.

The local refiner substituted *must .. be* instead of *is* for emphasis and used *it* to avoid redundancy.

The following example does not contain implication:

Example 4.9

```
(Forall x/EspritProject) (Exists y,z/Integer)
(Exists t1,t2/TimeInterval) (Exists t3,t4/TimeInterval)
(projMeetingNo(x,y,t1,t2) and projYears(x,z,t3,t4)
 and y<= z*4)
```

The first predicate generates,

y is the projMeetingNo of an EspritProject x.

The second predicate generates,

z is the projYears of an EspritProject x.

The mathematical relation generates,

y must be atmost four times z.

By substituting y and z, the final sentence is:

The projMeetingNo of an EspritProject must be atmost four times the projYears of it.

Let us consider one example with the if-then structure:

```
Example 4.10
    (Forall x/Student) (Forall y/Integer)
    (Forall t,t2,t3/TimeInterval)
    (credits(x,y,t1,t2) and y>=90 ==>
            academicLevel(x,Senior,t3) )
```

The definition of y is:

y is the credits of a student x.

The mathematical relation is,

y is at least 90.

Together they generate,

The credits of a student is at least 90.

The consequence is,

x's academicLevel is Senior.

The overall assertional sentence becomes,

If the credits of a student is at least 90 then his/her academicLevel is Senior.

4.6 Architecture: Sentence Generator

In this section we recapitulate the modules of the sentence generator with their functionality. The overall architectural view was given previously in Figure 4.1.

4.6.1 Recognizer

Input: Object definition.

Output: Identified components.

Function: The recognizer is a parser that identifies the components of an object definition. It also identifies the three top level components of an object: introductory, attribute and assertional components.

4.6.2 Arranger

Input: Assertional and attribute components.

Output: Components with re-ordered attributes; assertions moved to the end; tags.

Function: At first, the arranger moves all the integrity constraints and deductive rules to the end of the object definition, just before the END clause. Then, if the object is not a token, it re-orders the attribute categories, along with the attributes associated with them, to ensure that identical categories are adjacent. After that, for each category the arranger groups the attributes having the same type together. It also generates tags to denote where the maximum adjacent categories occur. The tags are passed to the tag pool.

4.6.3 PSG

Input: Introductory component; re-ordered attribute component; assertions.

Output: Initial sentences; tags.

Function: If the number of attributes in a category is more than three then they are partitioned into groups of three. If the last group has only one attribute it is added to the previous group thus making it null. Each group participates in generating one sentence by using a phrase-structured grammar. The assertions are translated by the assertion handler as described in Section 4.5.

In general, built-in class types are ignored for a client audience. For introductory components, type-compatibility is checked to determine what kind of sentence is to be generated. The sentences generated by the PSG are later revised.

In the description of tokens, attribute category labels are generally not included in the sentence; only data and their descriptive attribute labels are taken into account, but in absence of attribute labels, category labels may participate in the sentence.

4.6.4 Local Refiner

Input: Initial sentences; temporal constraints; refining rules.

Output: Revised sentences.

Function: The initial sentence produced by the PSG are revised one-by-one by the local refiner. Moreover, phrases are constructed from temporal components as described in Section 4.4 and are incorporated in the sentences.

The refining tasks involve number and subject-verb agreement, capitalization, choosing articles, pronouns and case forms, phrase substituting, normalizing compound words and punctuation.

The article rule relating to a and an has already been described. The capitalization of a word is done at the beginning of a sentence, when an object is referred to as a class or an attribute (the elements of the object has the same name but in lower case) and for those object names that appear in capitalization list (e.g., Esprit Project). Number and subject-verb agreement rules follow standard English grammar rules. Article, case and gender rules were discussed in Section 4.3.1. The local refiner also splits compound words if they are not referred to as attributes and removes redundant clauses. In a sentence, attributes appearing in with a which-clause follow the ones that do not. While integrating temporal constraints, special care is taken to revise the tense of the sentence to conform with temporal relations.

While capitalization and normalization of words can be done anytime, phrase substitution must be done before applying rules of subject-verb agreement, articles, pronouns and case forms, and punctuation is done last.

4.6.5 Global Refiner

Input: Revised sentences; tags; text improvement rules.

Output: Description of an object definition.

Function: The primary responsibility of the global refiner is to put together the refined sentences in such a way that they appear as a well-organised coherent text. With that purpose, it adds appropriate connectives among sentences and descriptions of the various components of the object definition. It also makes proper pronomi-

nalization and introduces, as necessary, new sentences at tag points. As shown in examples in the previous sections, sometimes the global refiner combines sentences to improve readability and adds phrases right before assertions are stated, such as, The deductive rule is as follows: ... . Some of the rules dictating the choices were discussed in Sections 4.3.1, 4.3.2, and 4.3.3.

The rules mentioned in this chapter are by no means complete. It is expected that more rules will be formulated in the implementation phase.

4.7 Conclusion

In this chapter, we have shown how sentences are generated from an object definition with the help of the recognizer, arranger, PSG, local refiner and global refiner. This chapter demonstrates the functions of these components and information flow among them. Translation of temporal constraints as well as assertions were also discussed. With the process described in this chapter, a paragraph of text can be generated from a given object. But, nevertheless, it does not constitute the whole report which may contain descriptions of a set of objects. Determination of that set of objects is the topic of the next chapter.

Chapter 5

Discourse Strategies

In the previous chapter we saw how sentences were generated from a description of single objects, but we did not determine how to generate them from a pool of objects in the specification. This chapter addresses this issue and provides some methods for content selection and discourse strategy. It also briefly discusses input and output processing. The module specifications in this chapter, along with the ones in Chapter 4, constitute an overall architectural design for GETS.

5.1 Data Flow

Figure 5.1 shows the flow of information starting from a user request to the generation of final report as a response. The user request is processed by an *input processor* that determines what type of report is to be generated. GETS can handle three types of reports, i.e., full, selective and data reports, which are discussed in next section.

Requests for full and selective reports are fed into the *content selector* whose task is to pick appropriate objects relevant to the request and order them in a specific fashion. These chosen objects are then sent to the *sentence generator*, described in Chapter 4, where appropriate sentence-level descriptions are generated. The *output processor* then ensures cohesiveness of the inter-object descriptions.

A data request is delivered to a *query processor* which formulates the query and retrieves solicited data. The output processor formats the data in tabular form or, if

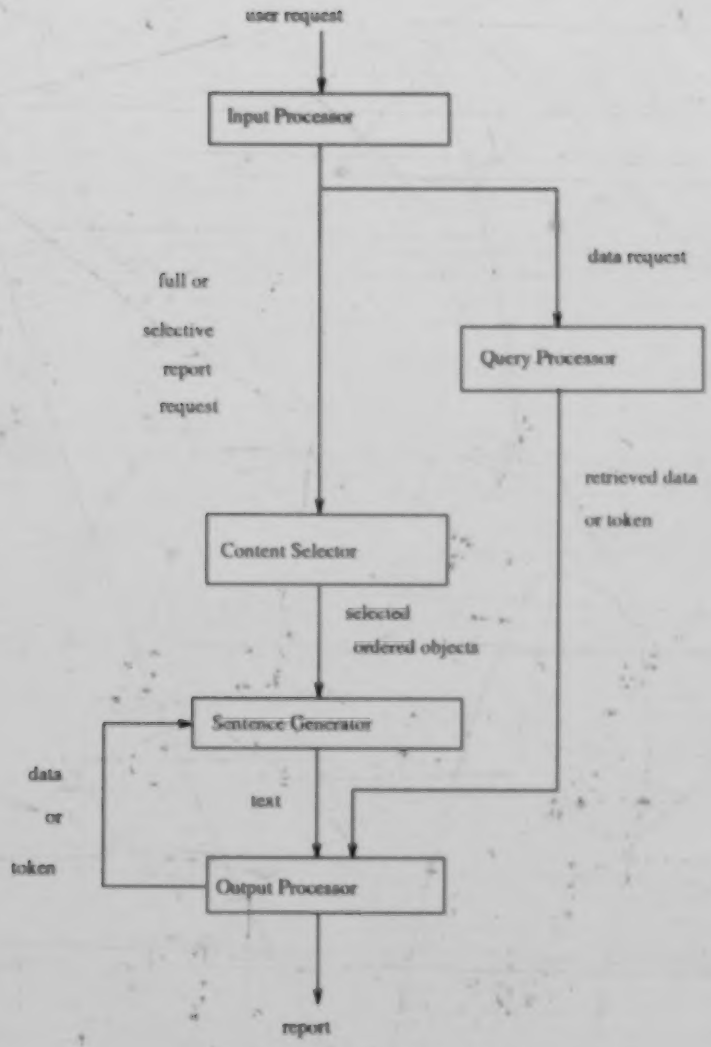


Figure 5.1: Data flow in GETS

the size of data is small, sends them to the sentence generator to describe them in natural language before forwarding to the user as a response.

Some implementations of Telos already have a query processor such as the one described in [Topaloglou and Koubarakis, 1989]. The translation from user request to queries is not considered as part of this thesis. In the subsequent sections we are going to discuss the input processor, the content selector and the output processor.

As it has been noted in [MacDonald, 1987], the generation process consists of goal identification, content selection and rhetorical planning, and syntactic realization. In our system, the input processor helps in identifying user goals while the content selector performs selection and planning. The sentence generator described in the previous chapter is responsible for further optimization and realization. But since the sentence generator only provides individual descriptions of objects, we have added an output processor that ensures these descriptions are not disconnected but that a continuous integrated description is achieved.

5.2 Input Processor

In support of the objective of understanding a formal specification, GETS entertains three types of user questions:

a) full report,

b) selective report, and

c) data report.

The user requests *full report* when (s)he wants a description of the whole specification. This report describes objects, but not necessarily in the same physical order in which they were specified. As well, meta-attribute class definitions are avoided because they are often too complicated to understand in natural language due to their abstract nature. The token definitions, too, are largely discarded in this report. They are mentioned merely to illustrate instances for objects.

Selective reports are partial reports generated when user is interested only in a

subset of the object definitions. The user need not explicitly mention all the objects (s)he wants to be described by the content selector. However, (s)he must specify key objects that he or she is interested in.

The third type of report, *the data report*, is a report generated in special circumstances that produce token-level data. An example is a report generated in response to an instructor's request for a list of students in his/her class. Telos has a built-in query language that is used to answer these requests for token-level data and the response is put in a tabular form with one or two introductory sentences. Another need for data reports arises when the user requests some examples of ground instances that are described in natural language.

The input mechanism of the current version of GETS is by menu selection with limited user input in specifying objects and context selection. A natural language query interface is beyond the scope of this thesis, but it, nevertheless, could easily be integrated with the current system.

5.3 Content Selector

The objective of the content selector is to retrieve necessary objects from specification to meet the goal of a user request. The strategies involved are described in this section.

5.3.1 Full Report

To meet the user request for a full report an appropriate order of objects is generated. The ordering is important to make related object descriptions appear integrated. This *locality* of information helps maintain the reader's focus in a description. In a Telos specification a strict ordering is not imposed by the language other than that an object must be defined before it is referenced. This simple ordering is inadequate for generation because related object definitions largely appear scattered in the specification.

In a full report all abstract objects, except for meta-attribute class definitions, are



Figure 5.2: Membership Link

described. Tokens are described selectively. Before we describe the ordering technique of objects, we give a few definitions that will be used subsequently:

Definition # 1: Membership Link

A membership link is represented as a solid directed arc indicating that one object is a member of a class as specified in the IN clause. For example, IndividualClass A IN B is represented graphically as in Figure 5.2.

Definition # 2: Specialization Link

A specialization link is represented as a double arrowed arc depicting a specialization relationship as described in an IsA clause. For example, IndividualClass A IsA B is represented as in Figure 5.3.

Definition # 3: Token Link

A token link is represented as a broken lined arc denoting ground instances in the specification knowledge base. For example, Individual A IN Token, B is represented as in Figure 5.4.

Definition # 4: Referential Link

A referential link is represented as a dotted arc denoting to which objects another

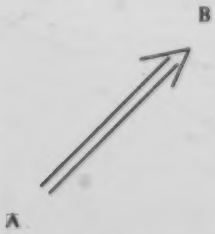


Figure 5.3: Specialization Link



Figure 5.4: Token Link

object refers. These links result from attribute component of the object definition. For example, the following definition is depicted in Figure 5.5.

```
IndividualClass A IN B
WITH
    attribute_1 : C;
    attribute_2 : B
END;
```

Tokens do not have referential links.

Definition # 5: Center of Activation

In a graph, a node about which information has been requested explicitly by the user is called a *center of activation* for the user request. A center of activation is denoted by two concentric circles around the node in a graph.

Definition # 6: Activated Nodes

In response to a user request the content selector highlights the potential nodes to be described. The nodes are known as *activated nodes*. In the graph, activated nodes are denoted by a circle around them.



Figure 5.5: Referential links

We now discuss full report generation techniques. The ordering technique involves three steps as discussed below:

a] *Graph Representation:* A pass is made on the specification to generate a graph denoting the inter-relation of the objects. The graph contains abstract objects and tokens (leaf-nodes). The graph generation algorithm is given in Figure 5.6. Statements have been numbered to facilitate referencing in the following discussion.

Objects are processed one at a time. Links are created from the object being processed to the other objects that appear in the IN and IsA clauses and as attribute types. In creating a link, if a node object does not exist then that node is generated first. The existence test is done at statements 3 and 8. We start with a null graph and at the end of applying this algorithm we have generated the graph that may contain a few disjoint subgraphs. If an IN or IsA clause does not exist, statements 4, 5 and 6 return null, i.e., they do nothing.

b] *Graph Simplification*: The graph generated by the above process may contain cycles, and multiple links may exist for a single object. This may result in multiple description of the same object if we follow the links when choosing objects to describe; therefore, the graph is simplified to eliminate redundant links as much as possible and thus largely reducing the graph to a tree. Referential links are kept intact; they are only used when the user wants referred objects to be described.

The simplification process employs a set of heuristics. Below we give three heuristics that help in simplification of the graph. These heuristics are used by the algorithm Graph_Simplification that is given subsequently.

5.3.1.1 Membership *or* Specialization Link Reduction Heuristic

If any object contains only membership links or only specialization links to multiple objects, all but one link are marked *inactive*. In doing so, links to built-in classes (e.g., S_Class, M1_Class etc.) are marked first. If there still remains more than one link, all but one (arbitrarily) are marked inactive. As an example, in Figure 5.7(a)

```
Procedure Graph_Generation(Specification);

begin

  1: for each object Oi in Specification do

     begin

       2: for each object Oj in the introductory component do

          3: if Oj is not found in the graph then
                create a node containing Oj;

       4: create membership links from Oi to the members
             of IN clause;

       5: create specialization links from Oi to the members
             of IsA clause;

       6: if Oi is a token then
             create token links from Oi to the members of IN clause;

       7: for each attribute, Ai  in  Oi do

          begin

          8: if  <attribute-type>, Ti of Ai does not exist as a
             node in the graph then
                create a node representing Ti;

          9: if a referential link does not exist between  Oi
             and Ti then
                  create a referential link between them.

          end;

     end;

end;
```

Figure 5.6: Algorithm for Graph Generation

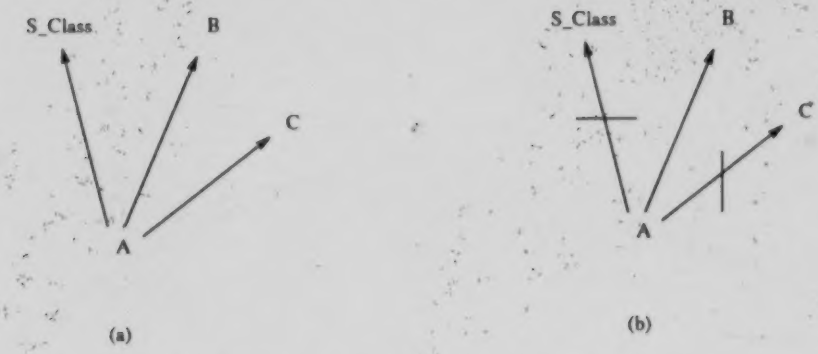


Figure 5.7: Marking links Inactive: 1

the links between A and S_Class, and A and C are marked inactive, as shown in Figure 5.7(b).

5.3.1.2 Membership *and* Specialization Link Reduction Heuristic

If an object has membership links as well as specialization links then all membership links are marked inactive. This is because we prefer to keep specialization links since specialization relationships result in attribute inheritance. Now, if there are multiple specialization links the resolvement is made according to the *Membership or Specialization Link Reduction Heuristic* given before. In Figure 5.8(a) only the links from A to B and E are specialization links, and links from A to C and D are

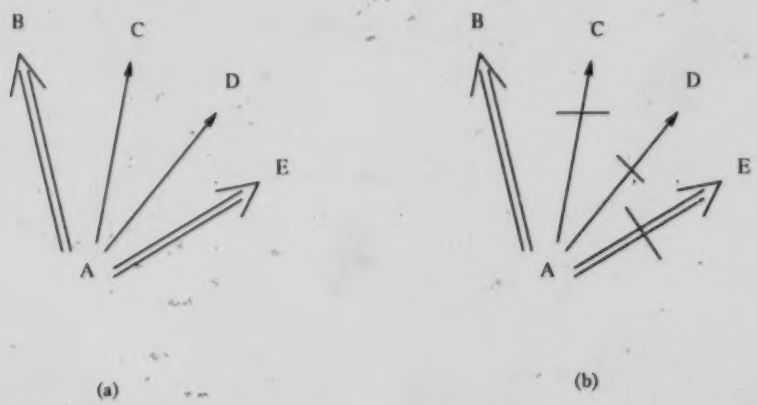


Figure 5.8: Marking links Inactive: 2

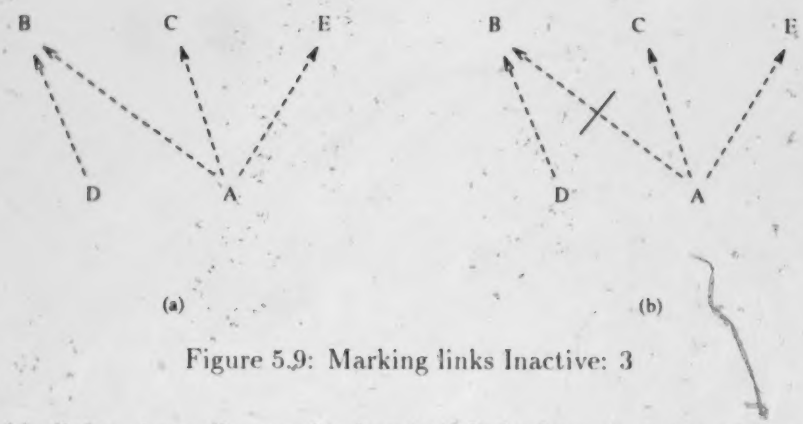


Figure 5.9: Marking links Inactive: 3

membership links. According to this heuristic links from A to C and D are marked inactive since they are membership links, and of the two specialization links only A to B remains active, as shown in Figure 5.8(b).

5.3.1.3 Token Link Reduction Heuristic

For token links, a test is made to determine whether marking a link leaves a parent node with no active token. In such a case that link is kept active; otherwise the *Membership or Specialization Link Reduction Heuristic* is applied for token links. For example, in Figure 5.9(a) D and A are two tokens that are ground instances of B, and A is also a ground instance of B, C and E. Since A has multiple links, we need to mark some of them as inactive. The link from A to B can be safely marked but links from A to C and from A to E can not because that would leave objects C and E isolated with no active token. Therefore, the resulting links are as shown in Figure 5.9(b).

The algorithm that simplifies the graph by marking is given in Figure 5.10. The marking is not permanent; that is, the marked links may be re-activated, if necessary, as deemed by the selective report or data report procedures.

c]*Graph Traversal*: The last step in ordering is generating a traversal order in which object definitions will be passed to the sentence generator. The graph may be disjoint before or after the marking process. If it is disjoint, the root nodes of subtrees

```
Procedure Graph_Simplification(Graph)

begin

  for each node in the graph which has multiple
       emanating links do

  begin

     apply Membership  or Specialization  Link Reduction Heuristic;

     apply Membership  and Specialization Link Reduction Heuristic;

     if it is a leaf-node then
          apply Token Link Reduction Heuristic;

  end;

end;
```

Figure 5.10: Algorithm for Graph simplification

are put in a list. The simplified graph can be traversed in depth-first, breadth-first or any other manner. If we ignore token links the simplified graph becomes a tree after applying the heuristics given, but with token links it may not necessarily be a tree. We present here one traversal scheme that we believe provides a coherent organization for text generation.

From the disjoint root list each node is taken and the corresponding subtree is processed. Each subtree is traversed depth-first except when a node has one or more specialization link(s) to its children. In this case a subtree is traversed in a breadth-first manner, with specialization children preceding membership children. This rule is recursively applied for each subtree. A token link preempts these rules. If a node has token link(s) those leaf node(s) are visited immediately after that node. If a token is reached for which two or more links exist all but the current parents link are specially marked. Later when traversing this specially marked link this token is

cited as an example without elaboration since it has already been described. For a node, not all its leaf nodes are expanded if full or selective report is desired. Now, the built-in nodes are deleted from the list of traversed nodes. This is done because built-in classes are not considered in our natural language expansion.

The advantage of the traversal described above is that locality of information is well maintained. For example, if **Employee** is a class and **Researcher** and **Secretary** are specializations then the above traversal technique describes **Employee** followed by **Researcher** and **Secretary**. Since they are all of the same object 'type', the reader finds not only an 'overall picture' but can also refer back to the appropriate place later if necessary since the the descriptions of these objects are likely to appear in the same section. Thus the locality of information is achieved and consequently organization of the text is improved.

The algorithm Graph_Traversal, as shown in Figure 5.11, makes use of data structures that we call *depth-queue* and *breadth-queue*. They are basically queues that hold nodes of the graph as elements. The pictorial representation of depth- and breadth-queues are shown in Figure 5.12. As in a regular queue, elements are deleted from the front and added at the rear. A The difference between these two types of queues is that only a breadth-queue allows an element to by-pass the elements in front of it to exit from the queue. This feature is needed during re-arrangement of queue elements. Each element may contain a pointer that refers to another depth- or breadth-queue and this element is known as **parent element** of the referred queue. When an element is deleted its associated pointer does not get deleted. The function **parent(queue)** returns a pointer to a queue which is the parent of this **queue**. If no pointer is coming in, then **null** is returned.

In Figure 5.12, x is a depth-queue with elements A, B and C, and y is a breadth-queue with elements E and F. Element A is pointing to the breadth-queue, y and therefore, A is the **parent element** of y. The function **parent(y)** returns a pointer to x, and **parent(x)** is **null**.

```
Procedure Graph_Traversal(graph);
var
   current_node   : node;
   current_Q      : pointer;
begin
     place the root in a depth-queue;
     current_node := root;
     current_Q    := this queue;
     visit(current_node);
 5:  if current_node has spec. link(s) then
        begin (* process nodes that has some specialization children *)
             put its non-token children in a breadth-queue;
             make a pointer from current_node element in the queue
                    to this breadth-queue;
             move the elements with mem. links after the elements
                     with spec. links;
             visit(elements in the breadth-queue in that order);
             current_Q    := this queue;
             current_node := front(current_Q);
             goto line 5;
        end
 6:   else if current_node has only membership link(s) then
        begin (* process node that has only membership children *)
             put its non-token children in a depth-queue;
             make a pointer from current_node element in the queue
                    to this depth-queue;
             current_Q    := this queue;
             current_node := front(current_Q);
             visit(current_node);
             goto line 5;
        end
 7:   else
        begin (* process leaf nodes or nodes that has only tokens *)
             delete current_node from current_Q;
        10:  if current_Q is empty then
                  begin
                      current_Q := parent(current_Q);
                      delete parent element, if any;
                  end;
             repeat line 10 until current_Q is non-empty or null;
             if current_Q=null then
                exit to end;
             current_node := front(current_Q);
             if current_Q is a depth-queue then
                visit(current_node);
             goto line 5;
         end;
end;
```

Figure 5.11: Algorithm for Graph Traversal

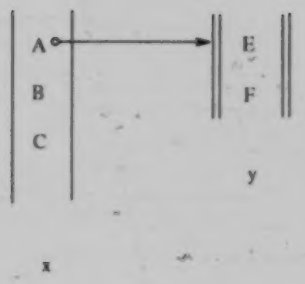


Figure 5.12: An Example of depth- and breadth-queues

```
Procedure visit(node and its tokens);
begin
     include this node in the output list;
     if this node has an unmarked token link then
       begin
            include that token in the output list;
            mark other links of this token, if any;
       end
      else if this node has a marked token link then
            include that token in parenthesis in the output list;
end;
```

Figure 5.13: Procedure Visit

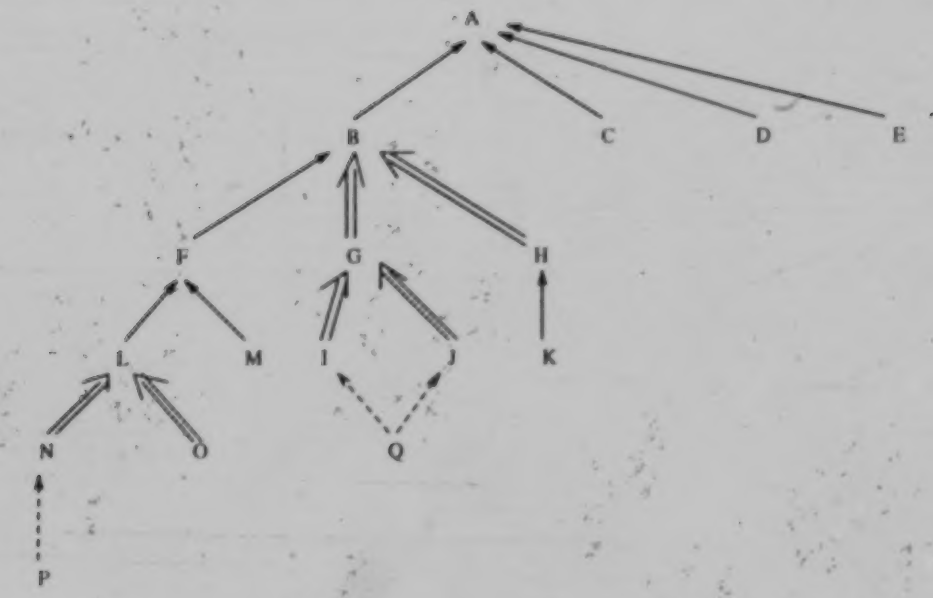


Figure 5.14: A graph for traversal

In Algorithm Graph_Traversal current_node is the node under process and current_Q is a pointer to the queue. This algorithm calls Procedure visit in Figure 5.13 that includes the node and its tokens, if any, in a global ordered list that is the output. Algorithm Graph_Traversal is called for each disjoint subtree. After all subtrees have been processed, the built-in classes are deleted from the output list.

5.3.2 Example of a Graph Traversal:

As an example of the graph traversal technique just described, we consider the simplified graph in Figure 5.14. In this graph, nodes A to O are high level objects whereas P and Q are tokens. Nodes G, H, I, J, N and O are specializations of other classes of objects.

The traversal starts at root node A and using a depth-first scheme the next node picked is B. But since B has two specialization nodes the traversal for the next level of the subtree rooted at B is breadth-first. Again, since nodes G and H are specializations they are traversed before node F. Since G has two specialization nodes, I and J, they

are visited next with the token, Q, being visited right after I. While visiting Q, the link from Q to J is specially marked. During the visit of J, Q is not traversed again but, nevertheless, is mentioned as an illustrative instance. The next node visited is K.

Now, the subtree rooted at F is traversed depth-first. The next nodes visited are L, N, P, O, and M, in that order. Since P is a token it is visited immediately after N. Finally, the nodes C, D, and E are visited.

The overall traversal is:

[A B G H F I Q J (Q) K L N P O M C D E].

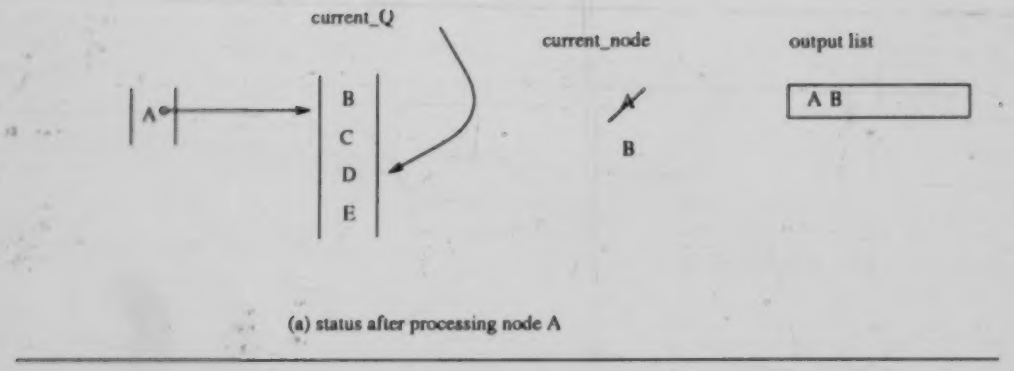
The node in parenthesis implies that it is not actually expanded in description.

We will now show, by tracing, that the algorithm actually produces the same list. First, root A is placed in a depth-queue and current_Q points to this queue and current_node is A. Since A has only membership links, according to statement 6 in the algorithm Graph_Traversal, a depth-queue is created with elements B, C, D and E in it. At this point, the snapshot of data is as in Figure 5.15(a).

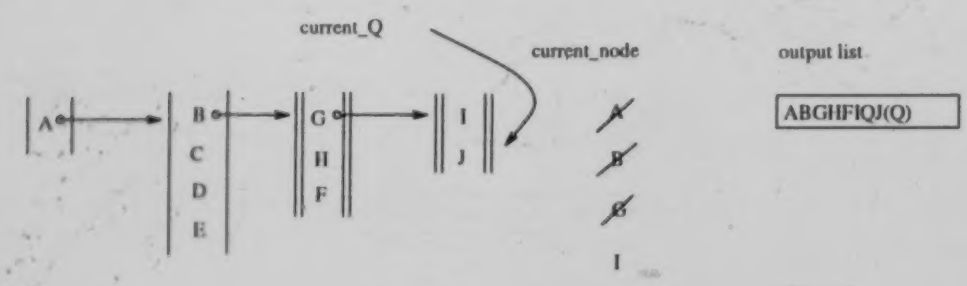
Since node B has specialization links, according to statement 5 in the algorithm, the children of B are put into a breadth-queue and F, being a membership node, is moved behind specialization nodes G and H in the queue. G is now the current node. The processing of node G generates a breadth-queue with I and J in it. At this time, the snapshot is as in Figure 5.15(b). Next, I and J are processed, their queue vanishes, G is deleted and H is being processed, as shown in Figure 5.15(c). In this fashion, the algorithm proceeds to generate the final list that was given before.

5.3.3 Example of a Full Report

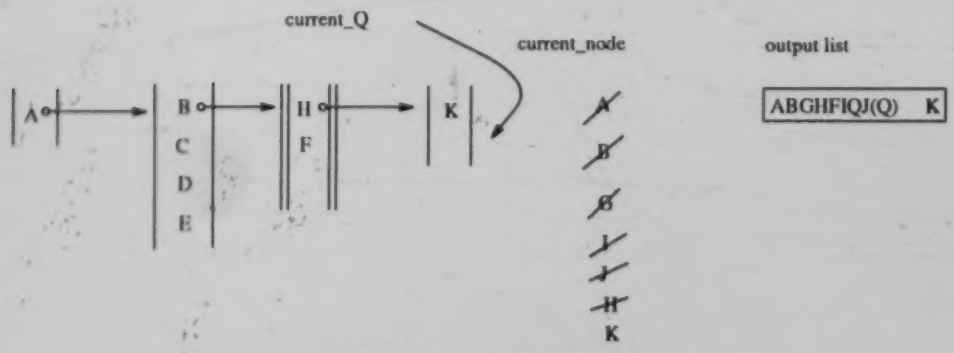
We now return to the object definitions of a Department Information System in Chapter 3 (Examples 3.2 through 3.11, and Figure 3.1). In this subsection we will generate an ordering for that partial specification by tracing the algorithm given before.



(a) status after processing node A



(b) status after processing node G



(c) status after processing node H

Figure 5.15: Snapshot of Data During Execution of Alg. Graph_Traversal

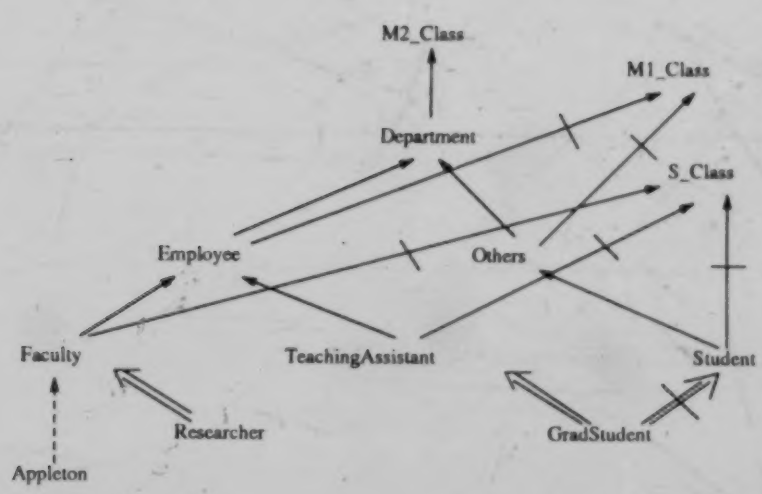


Figure 5.16: Graph generated from Figure 3.1 and its simplification

At first, a graph is generated with the three kinds of links described before. The graph is given in Figure 5.16. (Referential links are not shown in order to simplify the graph).

In the simplification stage, the nodes Employee and Others have two membership links to meta-classes. These links are marked *inactive*. For the same reason, the links from nodes Faculty, TeachingAssistant and Student to S_Class are marked inactive. For the node GradStudent the right specialization link is marked too.

The traversal begins at the root, M2_Class. In depth-first fashion the nodes Department, followed by Employee, followed by Faculty are visited. The node Faculty has a token link, therefore Appleton is visited immediately after Faculty. The node Faculty also has a specialization node Researcher therefore a switch is made from depth-first to breadth-first. (But as it happens, in this example both depth-first and breadth-first traversals are equivalent since it has only one child, beside the token). The node Researcher is visited next. The rest of the traversal order is:
TeachingAssistant, GradStudent, Others and Student.

The overall traversal produces the following list:

[M2_Class, Department, Employee, Faculty, Appleton, Researcher, TeachingAssistant,

GradStudent, Others, Student].

After deleting the built-in class M2_Class the final list is:

[Department, Employee, Faculty, Appleton, Researcher, TeachingAssistant, GradStudent, Others, Student].

5.3.4 Selective Report

If the user wishes to have a natural language text output of part of a specification, a selective report is generated. The user must provide, as input, one or more object nodes of interest. The graph, as mentioned in Section 5.3.1, can be generated once and used repeatedly for every user request. While the graph remains constant some of the links get activated or de-activated (as a result of marking) for different reports. For a selective report, user input is accepted and the nodes containing the requested objects are searched for in the graph. Once they are identified all links going into and out of these nodes are activated, except that parent nodes that have membership links to the nodes of centers of activation are not activated. All tokens of activated nodes are activated, too. We will illustrate this with an example in the next section.

Once all the links of all specified nodes have been activated the resulting subgraph becomes the graph of our interest. Now, the algorithms Graph_Simplification and Graph_Traversal are applied on this graph to generate the output list of objects to be described. From the list generated built-in classes are eliminated.

In selective report, GETS' notion of context is that, by default, one parental level and one children level of the node of center of activation constitute the current context. But in the parental level, only parents that have specialization links to it are activated because the node of interest inherits attributes from them. If additional levels, or any new activated objects are wanted further explicit user input is required.

Because of the incomplete nature of selective reports, it is possible that some descriptions refer to some classes that are not described in the text generated. The descriptions of those objects, at user's request, may be obtained by tracing the refer-

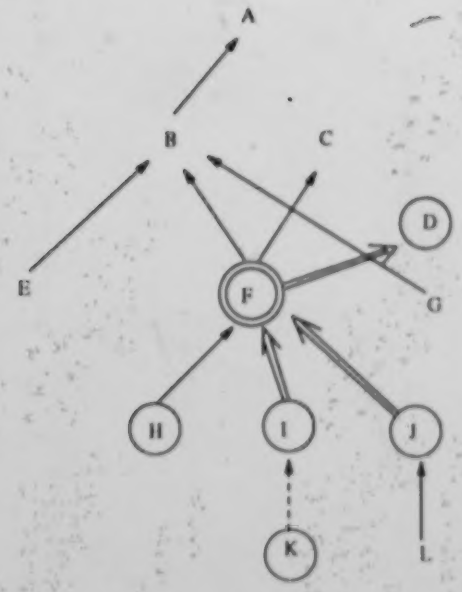


Figure 5.17: A graph from which a selective report is generated

ential links of the objects originally described.

5.3.5 Example of a Selective Report

Let us consider the graph in Figure 5.17. Suppose the user wants to have a selective report on the object F. First, the graph is searched for F, and then among its links (e.g., links FB, FC, FD, HF, IF, and JF) the links FD, HF, IF and JF are activated. The links FB and FC are not activated since they are membership links. The leaf-node K, being a token and attached to I, is activated, too. The circled nodes are the activated nodes, F being the *center of activation.*

In this case, the activated portion of the graph does not need simplification. The algorithm Graph_Traversal produces the following traversal order: [D F I K J H].

5.3.6 Data Report

Data reports contain descriptions of leaf-nodes (i.e., token information). If the tokens are selected, the object to which those tokens belong are visited. The leaf-nodes connected to that object node (i.e., the sibling tokens) are then retrieved. For example,

if a list of students is wanted the object Student is searched first then the tokens are visited by following the token links of Student.

If specific information (data) is requested a query is formulated by the Query Processor. The answer to the query is converted into a natural language response.

5.4 The Output Processor

A major task of the output processor is to provide top-level information about the tree and to format outputs. In particular it performs the following three things:

i]*Preamble adding*: By preamble we mean an overall description of Aobjects are given. At each node, before describing the node's children, one or more sentences are added that states who the children are and what types they are. Tokens are not mentioned. As an example, in Figure 5.14 at node A, a sentence like the following preamble may be added before any description is generated:

A includes the-classes B, C, D and E.

At node B this sentence may be added:

B has two specializations G and H, and a class F.

For a more concrete example, we consider Figure 5.16 in which the following may be introduced at node point Employee:

Employees are either faculty or teaching assistants.

Then faculty and teaching assistant may be described in turn.

For selective reports, the relationship of center object to other activated objects are explained in the preamble. For example, in Figure 5.17, we must state, in the preamble, that F is a specialization of D and the fact that F has two specializations I and J, and a regular class H.

ii]*Sectioning*: The presentation of the report can be enhanced by adding section headers and numbering. The root is level 0 and subsequent levels are numbered numerically, in increasing order. These are the section numbers while the nodes at each level are the headers for that section. Subsequent levels are divided into

subsections, enumerations etc. Tokens are excluded.

As an example, in Figure 5.16 the **Department** report is divided into two sections, Employee and Others. The section Employee has two subsections: Faculty and Teaching Assistant. The section Others has a subsection Student. The subsections Faculty and Teaching Assistant have enumerations of Researcher and Grad Student, respectively.

iii] *Data Formatting*: For data report, lists of data may be arranged in tabular form with the attribute names as column header and each tokens as rows.

5.5 Conclusion

This chapter presented a high level architecture of GETS. The issues of inputs, selections of appropriate objects and final report presentation were discussed. Three different types of reports were identified and procedures to generate them were described. The techniques discussed in this chapter, along with the techniques in Chapter 4, give a global picture of the activities of GETS.

Chapter 6

Comparisons

In this chapter, we compare our system, GETS, with another natural language generation system called GIST Generator. We chose GIST Generator because it also works on a specification domain. The specification language, GIST, has been described in Chapter 2, and example text produced by the GIST Generator has also been given in that chapter.

6.1 Comparison with GIST Generator

GIST Generator, introduced in Chapter 2, also produces natural language descriptions of a specification encoded in the specification language GIST. The difference between the GIST Generator and GETS largely arise from the differences in the features of the underlying languages upon which they operate. Both GIST Generator and GETS generate language from representations that were not originally designed for language generation and therefore, some information necessary for natural language generation is not captured in these specification languages. For this reason additional information, such as lexical type information, must be provided to the generator before it is run. A more detailed comparison of GIST Generator and GETS, based on their features and capabilities, follows:

1. The specification language GIST is simpler. It consists of a set of type definitions, agents and actions. Telos, on the other hand, is much more complex

with its first-order assertions, temporal constraints and meta-attribute definition capabilities. (In addition, it has reasoning and querying facilities.) As a consequence, the language generation is much more complicated in Telos than in GIST. For Telos, different generation strategies have to be taken for different parts of the language, e.g., assertions are handled quite differently than the object definition parts. A significant difference between the two languages is that GIST has explicit actions whereas Telos has actions that are implicit in activity classes. Therefore, while we can have a procedural description of some activity in GIST, we cannot have the same in Telos. The only inference mechanism in Telos is deductive rules. The advantage of procedural descriptions is that they are more action-oriented and more comprehensible in the context of a specification. As a result, part of the specification where actions are being described would be more lucid in GIST than in Telos.

2. GIST Generator has a limited number of relationships between attributes and their types and as a result, uses a set of mappings to generate certain English sentences easily. But in Telos the possible relationships between an attribute and its type are virtually unlimited.

   In particular, GIST has only three types of attribute relations: a noun form, a 'part-of' form and a verb form. GIST Generator requires user input to distinguish among them. These attribute relations can be translated into three types of sentences:

   e.g., type ship(Destination|port);
   is translated into:

   Each ship has a destination which is a port.

   type auto(Engine|engine);
   is translated into :

   Each auto has an engine.

type pier(Handle|cargo)

is translated into:

Each pier handles a cargo.

In Telos attribute relations are vast and sometimes they contain compound words which make them difficult to translate in a few pre-determined way.

3. In the case of the GIST Generator, minimal input from the user is necessary. Only lexical types of words (e.g., noun, verb etc.) have to be provided. As described before, the number of relationships between types are limited and these can be easily indicated by the user. But in GETS the user has to identify a greater number of relationships among attributes and their types. GETS provides a greater degree of flexibility in sentence generation, but to accomplish this it needs more user input since not all of the language generation information is captured in Telos. For example, the lexicon in GETS, which contains gender, verb forms of words and other lexical information, is more complex. Once again, since actions are encapsulated as attributes in Telos, they have to be distinguished from other attributes. In GIST actions correspond to verbs with six specific cases, and hence, they are easier to translate.

4. Telos is object-oriented, the objects being arranged in class hierarchies. As a result, GETS has to handle inheritance, classification/instantiation, aggregation/decomposition and generalization/specialization. This leads to the use of graph generation techniques to capture the inter-relationships of objects. GIST Generator does not use any graph generation.

5. GIST Generator produces a full report only. The content selector and ordering mechanisms, therefore, are primitive. Selective report and data report are not available in GIST Generator. On the other hand, GIST Generator can explain a symbolic trace of the specification [Swartout, 1983]. GETS has no such

capability.

6. The preamble in GIST Generator is simple, appears only at the beginning of the report and does not describe at every section level of the report. GETS has a rather sophisticated preamble system as well as a sectioning feature that enhances the presentation of the report.

7. As pointed out in [Swartout, 1982] GIST Generator makes decisions largely based on local information. In contrast, GETS has a global view of the whole specification, can make focused explanations, and can perform optimization based on its global view.

Chapter 7

Conclusion

In this concluding chapter we concentrate on two aspects: first, we recapitulate our research, drawing conclusions from it and identifying our contributions; and secondly, we identify a number of future research issues related to our work. While discussing future research, we point out the limitations of GETS and discuss other areas to which our strategies may be extended.

7.1 Summary and Conclusion

Chapter 1 of this thesis establishes the objective of our research which is to generate natural language descriptions of a formal specification in Telos. The chapter provides the motivation by discussing the goals of a formal specification and how a generated report might help achieve these goals. Our system, GETS is intended to generate explanations of an encoded specification in a report format.

Previous research in natural language generation was reviewed in Chapter 2. We concentrate on three groups of language generators: general generators, generators from predicate calculus, and generators from a specification. This chapter outlines how research in language generation has developed and the current state of research in this area. It also overviews briefly the features of GIST, a specification language, from which GIST Generator produces natural language descriptions.

Chapter 3 introduces Telos, a requirement modeling language for specifications

which is the underlying representation for our generation system. Telos has object-oriented properties and is based on semantic net and frame-based concepts. It also integrates a first-order predicate language as assertions. Chapter 3 gives the syntax and semantics of Telos.

The architectural design of the sentence generator is the content of Chapter 4. The Sentence Generator component of GETS has five modules: the recognizer, arranger, preliminary sentence generator, local refiner and global refiner. The recognizer is a parser and the arranger is an optimizer. The preliminary sentence generator produces initial sentences that are later revised by the local and global refiners. The two refiners use a set of rules to perform their activities. The strategies of the optimizer and the rules of the refiners given may be extended to other object-oriented generation systems.

We have employed an incremental strategy of text production. At the lowest level, individual sentences are generated and revised. At the next level, some of these sentences are grouped together to create paragraphs and revised again to ensure coherence among sentences in a paragraph. These revisions are performed by the local and global refiners, respectively. At the third level, paragraphs are put together and preambles are added to furnish the final report. This part is accomplished by the Output Processor.

Chapter 5 gives the overall design of GETS which is divided into four main modules: the input processor, content selector, sentence generator and output processor. The input processor determines which of the full, selective, or data reports is wanted by the user. We derived a set of algorithms for the content selector which provides mechanisms for graph generation, simplification and traversal. Based on the concepts of queues, we introduced two data structures, depth-queue and breadth-queue that help in the traversal of the graph. The algorithm provided may be adapted to other object-oriented languages and hence is a contribution towards a language generator for object-oriented systems.

Chapter 6 compares our system GETS with the GIST Generator which also operates on specifications. The specification language GIST is simpler than Telos. GETS has sophisticated content selection and discourse strategies and preamble features. GETS also provides three different types of reports. On the other hand, the GIST Generator requires minimal user input.

The sentences that GETS is capable of generating are implementation dependent. The examples that were given in this thesis contain some technical words such as 'attribute' and 'specialization'. in English language descriptions of objects. These may be eliminated in the implementation phase, but to accomplish this additional input is necessary to identify relationships among attributes and their types and objects.

Object-oriented languages and databases are becoming increasingly popular in the 1990's. The strategies described in this thesis can be extended or adapted to generate natural language from other object-oriented systems as well. This is largely because we made use of the object-oriented properties and the modules are general enough to handle them. For a specific application, the grammar module has to be modified to accommodate generation of sentences specific to that domain.

The architecture of GETS is quite general and this helps in transporting these modules to other domains. The activities associated with the modules Input Processor, Content Selector, Sentence Generator and Output Processor are necessary for any generator. Undoubtedly a given generation would require different implementations of these modules based on variances in object definition and representation.

As mentioned before, GETS is currently being implemented in C++ to run under the Unix operating system. It is being implemented in cooperation with a NSERC summer student working for the Software Engineering Research Group.

7.2 Future Research

In the following, we discuss the limitations of GETS and how its techniques may be extended to other systems:

1. The current version of GETS has a menu-driven user interface. The task of the Input Processor is simplified by this means. For future research, a natural language interface to GETS may be considered. This would allow the user to state his/her questions in English instead of choosing options from a menu. The Input Processor will be a natural language analysis module that determines what kind of report the user wants. It will also communicate with the user in natural language to ask for inputs such as specifying a few key objects for a selective report. In this thesis, we concentrated on natural language generation, as opposed to language analysis, leaving development of a natural language interface for future. This interface should also be able to handle queries posed in natural language to generate data reports.

2. The data report generation capability of GETS is not complete. The translation of user's data request into the query language of Telos is beyond the scope of our research. It is not clear what form the output of the query processor is so that we can use this output form as a base-representation for translating the output into English sentences. However, data reports that require descriptions of tokens can be handled by the current system.

3. In dealing with assertions, the current version of GETS works well when all variables are either existentially or universally quantified (after applying reduction rules). But a mixture of existential and universal quantifiers sometimes requires careful selection of phrases, and aspects that has not been explained in this thesis. Quantifier problems constitute a separate area of research that has been investigated by other researchers. It would be interesting to see how

these problems can be applied in a Telos environment.

In Telos assertions, there are many temporal variables. Although these variables are necessary during temporal reasoning, they are very often redundant in natural language descriptions of the assertions. Sometimes (in our view, in most cases) the temporal variables can be ignored. But it is not clear when it is safe to ignore temporal variables with losing information. This issue requires further exploration.

4. GETS considers only TELL commands. The querying commands RETRIEVE and ASK, and update commands RETELL and UNTELL have not been taken into consideration. A natural language description of a query expressed in RETRIEVE or ASK command may be used as validation of the intended query. The description of RETELL and UNTELL may provide update information of the knowledge base. Therefore, GETS may be extended to process these commands.

   Although current techniques may be applied to processing RETELL and UNTELL commands, the querying commands may require different translation mechanisms than we have given, since they require the query language of Telos to be translated into English.

5. In this research, we have not considered meta-attribute definitions. They are included in Telos to explain the category labels. For example, the category label 'single' may be defined by a meta-attribute definition that puts an integrity constraint on this category label on its singleness. We believe that this explanation is not vital for the client because they only impose restrictions on some attributes which are used by Telos as a consistency checker. Again, because of their highly technical nature, the natural language descriptions of meta-definitions are not easily understandable by non-experts such as clients, and this issue could be examined in the future.

6. A problem faced during this research was that there was no fixed set of syntax and semantics for Telos. Telos is still under development and some new syntax may be developed in future. These new structures and semantic variations may require new techniques in translation in the development of future versions of GETS.

7. Our research was directed to a user group who are clients. The reports that GETS can generate may be helpful to analysts as well. However, analysts are generally experts, and the reports provided to them must be more technical than that of the clients. In our research, we did not address the issues of user modeling. Nevertheless, the content, depth of descriptions, and choice of words and sentence structures largely depend on what kind of audience receives a report. Therefore, user modeling is an important issue to examine. For example, future versions of GETS should be able to generate two types of reports for the same request – one report for an analyst and one for a client.

Bibliography

[Allen, 1981] Allen, J. F. An Interval Based Representation of Temporal Knowledge. In *Proceedings of the International Joint Conference on Artificial Intelligence*, Vancouver, B.C., 1981, pp. 221-226.

[Allen, 1983] Allen, J. F. Maintaining Knowledge about Temporal Intervals. *Communications of the ACM*, 26(11):832-843, November 1983.

[Allen, 1984] Allen, J. F. Towards a General Model of Action and Time. *Artificial Intelligence*, 23(2), July 1984.

[Allen & Hayes, 1985] Allen, J. F. and Hayes, P. J. A Common-Sense Theory of Time. In *Proceedings of the 9th International Joint Conference on Artificial Intelligence*, Los Angeles, CA, 1985, pp. 528-531.

[Appelt, 1981] Appelt, Douglas E. Planning Natural Language Utterances to Satisfy Multiple Goals. Ph.D. Dissertation, Dept. of Computer Science, Stanford University, 1981.

[Appelt, 1985] Appelt, Douglas E. Planning English Sentences. Cambridge University Press, 1985.

[Balzer et al., 1982] Balzer. R; Goldman, N and Wile, D. Operational Specification as the basis for Rapid Prototyping. In *Proceedings of the Second Software Engineering Symposium: Workshop on Rapid Prototyping*, ACM SIGSOFT, April, 1982.

[Bailes, 1986] Bailes, Alison J. Response Generation. M.Sc. Thesis, Department of Computing Science, University of Alberta, Edmonton, Spring, 1986.

[Borgida et al., 1985] Borgida, A.; Gallagher, J.; Mylopoulos, J. and Vassiliou, Y. CML: Conceptual Modelling Language. Tech Report, Institute of Computer Science, Research Center of Crete, 1985.

[Boyer & Lapalme, 1985] Boyer, M. and Lapalme, G. Generating Sentences from Semantic Networks. In *Natural Language Understanding and Logic Programming*, ed. V. Dahl and P. Saint-Dizier, North-Holland Publishing Co., Amsterdam, The Netherlands, 1985, pp. 181-189.

[Boyer & Lapalme, 1985a] Boyer, M. and Lapalme, G. Generating Paraphrases from Meaning-Text Semantic Networks. *Computational Intelligence*, 1:3 and 4, 1985, pp. 103-117.

[Chester, 1976] Chester, Daniel. The Translation of Formal Proofs into English. *Artificial Intelligence*, 7 (1976) 261-278.

[Clippinger, 1977] Clippinger, J. Meaning and Discourse: A Computer Model of Psychanalytic Speech and Cognition, Johns Hopkins Press, Baltimore, MD, 1977.

[Conklin, 1983] Conklin, E. Data-Driven Indelible Planning of Discourse generation Using Salience. Ph.D. Dissertation, Dept. of Computer Science, University of Massachusetts, 1983.

[Davey, 1979] Davey, A. Discourse Production. Edinburgh University Press, Edinburgh, U.K., 1979.

[Edgar, 1991] Edgar, Andrew. Natural Language Explanation of Natural Deduction Proofs. Extended Abstract, In *Conference Program & Abstracts of Papers*, IRIS-PRECARN First Annual Conference, Vancouver, B.C., June 12-14, 1991.

[Eherer et al., 1989] Eherer, S.; Jarke, M.; Miethsam, A. and Rose, T. A KBMS for Database Software Evolution: ConceptBase V.2.0 User Manual. Tech Report MIP-8936, Fafultat fur Mathematic und Informatik, Universitat Passau, Germany, November, 1989.

[Eherer et al., 1990] Eherer, S.; Jarke, M.; Miethsam, A. and Rose, T. ConceptBase V.2.1 — Addendum to: A KBMS for Database Software Evolution: ConceptBase V.2.0 User Manual. Tech Report MIP-8936, Fafultat fur Mathematic und Informatik, Universitat Passau, Germany, January, 1990.

[Gallagher & Solomon, 1986] Gallagher, John and Solomon, Levy. CML Support System. SCS Technische Automation und Systeme GmbH, Hamburg, June, 1986.

[Greenspan, 1984] Greenspan, Sol J. Requirement Modeling: A Knowledge Representation Approach to Software Requirement Definition. Ph.D. Dissertation, Dept. of Computer Science, University of Toronto, 1984.

[Greenspan et al., 1986] Greenspan, Sol J; Borgida, A. and Mylopoulos, J. A Requirement Modeling Language and Its Logic. In *On Knowledge Base Management Systems*, eds. Michael L. Brodie and John Mylopoulos, Springer-Verlag, 1986.

[Goldman, 1975] Goldman, N. Conceptual Generation. In *Conceptual Information Processing*. R. Schank (ed.), North-Holland/Elsevier, Amsterdam, 1975, pp. 289-372.

[Grishman, 1986] Grishman, Ralph. Computational Linguistics: An Introduction, Cambridge University Press, 1986.

[Hovy, 1985] Hovy, Eduard H. Integrating Text Planning and Production in Generation. In *Proceedings of IJCAI-85*, Los Angeles, CA, 1985, pp. 848-852.

[Hovy, 1987] Hovy, Eduard H. Generating Natural Language Under Pragmatic Constraints. Ph.D. Dissertation, Dept. of Computer Science, Yale University, March, 1987.

[Hovy, 1988] Hovy, Eduard H. Generating Natural Language Under Pragmatic Constraints. Erlbaum, Hillside, NJ; 1988.

[Hovy, 1988a] Hovy, Eduard H. Generating Language with a Phrasal Lexicon. In *Natural Language Generation Systems*, ed. D. D. McDonald and L. Bolc, Springer-Verlag, New York, 1988, pp. 353-384.

[Hovy, 1990] Hovy, Eduard H. Pragmatics and Natural Language Generation. *Artificial Intelligence*, 43 (1990) pp. 153-197.

[Jacobs, 1985] Jacobs, Paul S. A Knowledge-Based Approach to Language Production, Computer Science Dept., University of California, Berkeley, TR 86/254.

[Jacobs, 1987] Jacobs, Paul S. Knowledge-Intensive Natural Language Generation. *Artificial Intelligence*, 33 (1987) pp. 325-378.

[Jarke et al., 1988] Jarke, M.; Jeusfeld, M. and Rose, T. A Global KBMS for Database Software Evolution: Documentation of First ConceptBase Prototype. Tech Report MIP-8819, Fafultat fur Mathematic und Informatik, Universitat Passau, Germany, 1988.

[Kay, 1979] Kay, M. Functional Grammar. In *Proceedings of Fifth Annual Meeting of the Berkeley Linguistic Society*, Berkeley, Ca, 1979.

[Kemmerer, 1990] Kemmerer, Richard A. Integrating Formal Methods into the Development Process. In *IEEE Software*, Vol. 7, No. 5, September, 1990.

[Koubarakis, 1988] Koubarakis, Manolis. An Implementation of CML. Master's Thesis, Dept. of Computer Science, University of Toronto, 1988.

[Koubarakis et al., 1989] Koubarakis, M.; Mylopoulos, J.; Stanley, M. and Jarke, M. Telos: A Knowledge Representation Language for Requirement Modelling, Tech Report KRR-TR-89-1, Dept. of Computer Science, University of Toronto, January, 1989.

[Koubarakis et al., 1989a] Koubarakis, M.; Mylopoulos, J.; Stanley, M. and Borgida, A. Telos: Features and Formalization. Tech Report KRR-TR-89-4, Dept. of Computer Science, University of Toronto, February, 1989.

[Kukich, 1983] Kukich, Karen. Knowledge-Based Report Generation: A Knowledge Engineering Approach to Natural Language Report Generation. Ph.D. dissertation, Information Sciences Dept., University of Pittsburgh, 1983.

[Kukich, 1988] Kukich, Karen. Fluency in Natural Language Reports. In *Natural Language Generation Systems*, ed. D. D. McDonald and L. Bolc, Springer-Verlag, New York, 1988, pp. 280-311.

[Mann, 1982] Mann, William C. The Anatomy of a Systemic Choice. In *COLING-82*, ed. J. Horecky, North-Holland Publishing Co., 1982, pp. 195-200.

[Mann, 1983] Mann, William C. An Overview of the Penman Text generation System. *AAAI-83 Proceedings of the National Conference on Artificial Intelligence*, Aug. 22-26, 1983, pp.261-265.

[Mann, 1983a] Mann, William C. An Overview of the Penman Text generation System. Technical Report RR-83-114, Information Sciences Institute, University of Southern California, Marina del Rey, CA, April 1983.

[Mann, 1983b] Mann, William C. An Overview of the Nigel Text generation Grammar. Technical Report RR-83-113, Information Sciences Institute, University of Southern California, Marina del Rey, CA, 1983.

[Mann, 1984] Mann, William C. Discourse Structures for Text Generation. *Proceedings of COLING-84* , Stanford, CA, 1984, pp. 367-375.

[Mann, 1987] Mann, William C. Text Generation: The Problem of Text Structure. Technical Report RR-87-181, Information Sciences Institute, University of Southern California, Marina del Rey, CA, March 1987.

[Mann & Matthiessen, 1983] Mann, William C. and Matthiessen, C.M.I.M. Nigel: A Systemic Grammar for Text Generation. Technical Report RR-83-105, Information Sciences Institute, University of Southern California, Marina del Rey, CA, 1983.

[Maybury, 1989] Maybury, Mark T. GENNY: A Knowledge-Based Text Generation System. *Information Processing & Management*, V.25, No.2, 1989, pp. 137-150.

[McCoy, 1985] McCoy, K. Correcting Object-Related Misconceptions. Ph.D. Dissertation, University of Pennsylvania, 1985.

[McDonald, 1980] McDonald, D.D. Natural Language Production as a Process of Decision Making Under Constraints. Ph.D. Dissertation, Massachusetts Institute of Technology, MA, 1980.

[McDonald, 1983] McDonald, D.D. Natural Language Generation as a Computational Problem: An Introduction. In *Computational Models of Discourse*, eds. Michael Brady and Robert C. Berwick, MIT Press, Cambridge, Mass, 1983.

[McDonald, 1987] McDonald, D. D. Natural Language Generation. In *Encyclopedia of Artificial Intelligence*, ed. S. Shapiro, Wiley-Interscience, New York, 1987, pp. 642-654.

[McDonald, 1987a] McDonald, D. D. Natural Language Generation: Complexities and Techniques. In *Machine Translation*, ed. Sergei Nirenburg, Cambridge University Press, 1987, pp. 192-224.

[McKeown, 1982] McKeown, Kathleen R. Generating Natural Language Responses to Questions about Database Structure. Ph. D. Dissertation, Technical Report MS-CIS-82-5, Computer and Information Science Department, University of Pennsylvania, Philadephia, PA, May, 1982.

[McKeown, 1982a] McKeown, Kathleen R. The TEXT Generation System for natural Language Generation: An Overview. In *Proceedings of the Twentieth Annual Meeting of the Association for Computational Linguistics*, Toronto, Ontario, Canada, 1982, pp. 113-120.

[McKeown, 1983] McKeown, Kathleen R. Focus Constraints on language Generation. In *Proceedings of Eighth International Joint Conference on Artificial Intelligence*, Karlsruhe, Germany, August, 1983.

[McKeown, 1984] McKeown, Kathleen R. Using Focus to Constrain Language Generation. In *Computational Models of Natural Language Processing*, eds. B. G. Bara and G. Guida, Elsevier Science Publishing B. V. (North-Holland), 1984.

[McKeown, 1985] McKeown, Kathleen R. Discourse Strategies for Generating Natural-Language Text. *Artificial Intelligence*, 27 (1985), pp. 1-41.

[McKeown, 1985a] McKeown, Kathleen R. Text Generation: Using Discourse Strategies and Focus to Generate Natural Language Text. Cambridge University Press, 1985.

[Mylopoulos et al., 1990] Mylopoulos, J.; Borgida, A.; Jarke, M. and Koubarakis, M. Telos: A Language for Representing Knowledge About Information Systems. Dept. of Computer Science, University of Toronto, February, 1990.

[Sandewall, 1971] Sandewall, Erik. Formal Methods in the Design of Question-Answering Systems. *Artificial Intelligence*, 2 (1971), pp. 129-145.

[Scott et al., 1977] Scott, C; Clancy, W; Davis, R. and Shortliffe, E. Explanation capabilities of Production-based Consultation Systems. *American Journal of Computational Linguistics*, microfiche 62, 1977.

[Shapiro, 1981] Shapiro, S. C. Generalized Augmented Transition Network Grammars for Generation from Semantic Network. *American Journal of Computational Linguistics*, 8, 1981, pp. 12-25.

[Shieber et al., 1990] Shieber, S; Pereira, F; van Noord, G and Moore, R. Semantic-Head-Driven Generation. In *Computational Linguistics*, V.16, No.1, March 1990, pp. 30-42.

[Simmons, 1965] Simmons, R. F. Answering English Questions by Computer: A Survey. *CACM* 8, January 1965, pp. 53-70.

[Simmons & Slocum, 1972] Simmons, R. and Slocum, J. Generation from Semantic Nets, *Communication of the ACM*, Vol. 15, No. 10, 1972, pp. 891-904.

[Stanley, 1986] Stanley, Martin. CML: A Knowledge Representation Language with Application to Requirement Modelling. Master's Thesis, Dept. of Computer Science, University of Toronto, 1986.

[Swartout, 1977] Swartout, W. A Digital Therapy Advisor with Explanations. In *Proceedings of IJCAI-77*, 1977, pp. 819-825.

[Swartout, 1981] Swartout, W. Producing Explanations and Justifications of Expert Consulting Programs. Ph.D. Dissertation, Dept. of Electrical Engineering and Computer Science, MIT, 1981. (also Tech Report, TR-251, MIT Laboratory for Computer Science).

[Swartout, 1982] Swartout, Bill. GIST English Generator. In *AAAI-82 Proceedings of the National Conference on Artificial Intelligence*, August 18-20, 1982, pp. 404-409.

[Swartout, 1983] Swartout, Bill. The GIST Behavior Explainer. In *AAAI-83 Proceedings of the National Conference on Artificial Intelligence*, 1983, pp. 402-407..

[Tait, 1982] Tait, J. Automatic Summarising of English Texts. Ph.D. Thesis, Cambridge University, Dept. of Computer Science, 1982.

[Topaloglou, 1989] Topaloglou, Thodoros. An Extended Implementation of Telos. Master's Thesis, Dept. of Computer Science, University of Crete, 1989.

[Topaloglou & Koubarakis, 1989] Topaloglou, Thodoros and Koubarakis, Manolis. Implementation of Telos: Problems and Solutions. Tech Report, KRR-TR-89-8, Dept. of Computer Science, University of Toronto, May, 1989.

[Watt, 1988] Watt, Murray. The Realization of Natural Language with Pragmatic Effects. Master's Thesis, Dept. of Computer Science, University of Toronto, August, 1988. (also Tech Report CSRI-215, August, 1988).

[Weizenbaum, 1966] Weizenbaum, J. ELIZA - A Computer Program for the Study of natural Language Conversation Between Man and Machines. *CACM* 9, January 1966, pp. 36-45.

[Wing, 1990] Wing, Jeannette M. A Specifier's Introduction to Formal Methods. In *IEEE Computer*, Vol. 23, No. 9, September, 1990. pp. 8–24.

[Winograd, 1972] Winograd, T. Understanding Natural Language. Academic Press, N.Y., 1972.

[Wong, 1975] Wong, Harry K. T. Generating English Sentences from Semantic Structures. Master's Thesis, Dept. of Computer Science, University of Toronto, 1975.

[Woolf, 1984] Woolf, B. Context Dependent Planning in a Machine Tutor. Ph.D. Dissertation, University of massachusets, Dept. of Computer Science, 1984.